

# CSCI 32092 | Data Mining and Data Warehousing

## Assignment 02 - Project

### Step 1

Understand and review the dataset.

Deliverables:

- Data understanding summary
- Initial data quality report

### EXTRACT of ETL

```
In [1]: import pandas as pd

df = pd.read_csv('sales-data-samples.csv')
df.head()
```

```
/Users/subhashbandaraekanayake/anaconda3/lib/python3.11/site-packages/pandas/core/arrays/masked.py:60: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
```

```
from pandas.core import (
```

Out[1]:

|   | OrderDate                | Category        | City         | Country       | CustomerName  | Discount | OrderID        | P |
|---|--------------------------|-----------------|--------------|---------------|---------------|----------|----------------|---|
| 0 | 2011-01-04T00:00:00.000Z | Office Supplies | Houston      | United States | Darren Powers | 0.2      | CA-2011-103800 |   |
| 1 | 2011-01-05T00:00:00.000Z | Office Supplies | Naperville   | United States | Phillina Ober | 0.2      | CA-2011-112326 |   |
| 2 | 2011-01-05T00:00:00.000Z | Office Supplies | Naperville   | United States | Phillina Ober | 0.8      | CA-2011-112326 |   |
| 3 | 2011-01-05T00:00:00.000Z | Office Supplies | Naperville   | United States | Phillina Ober | 0.2      | CA-2011-112326 |   |
| 4 | 2011-01-06T00:00:00.000Z | Office Supplies | Philadelphia | United States | Mick Brown    | 0.2      | CA-2011-141817 |   |

5 rows × 28 columns

```
In [2]: df.shape
```

Out[2]: (9994, 28)

number of columns - 28

number of rows - 9994

```
In [3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 28 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   OrderDate                            9994 non-null   object
1   Category                             9994 non-null   object
2   City                                 9994 non-null   object
3   Country                             9994 non-null   object
4   CustomerName                        9994 non-null   object
5   Discount                            9994 non-null   float64
6   OrderID                             9994 non-null   object
7   PostalCode                          9994 non-null   int64
8   ProductName                         9994 non-null   object
9   Profit                              9994 non-null   int64
10  Quantity                             9994 non-null   int64
11  Region                              9994 non-null   object
12  Sales                               9994 non-null   int64
13  Segment                             9994 non-null   object
14  ShipDate                            9994 non-null   object
15  ShipMode                            9994 non-null   object
16  State                               9994 non-null   object
17  Sub_Category                        9994 non-null   object
18  DaystoShipActual                    9994 non-null   int64
19  SalesForecast                       9994 non-null   int64
20  ShipStatus                          9994 non-null   object
21  DaystoShipScheduled                 9994 non-null   int64
22  OrderProfitable                     0 non-null      float64
23  SalesperCustomer                    9994 non-null   float64
24  ProfitRatio                         9994 non-null   float64
25  SalesaboveTarget                    0 non-null      float64
26  latitude                            9994 non-null   float64
27  longitude                           9994 non-null   float64
dtypes: float64(7), int64(7), object(14)
memory usage: 2.1+ MB
```

This shows a pandas DataFrame summary of a dataset with 9994 entries and 28 columns. Here is a simple description of the columns and their data types:

1. OrderDate, Category, City, Country, CustomerName, Segment, ShipMode, State, Sub\_Category: These columns contain string data (objects).
2. Discount, Profit, Quantity, Sales, DaystoShipActual, SalesForecast: These columns have numerical data (integers and floats).
3. OrderID, PostalCode, ProductName: Object data type, likely strings.
4. Latitude, Longitude: Numerical data of type float.

```
In [4]: df.describe()
```

Out[4]:

|       | Discount    | PostalCode   | Profit       | Quantity    | Sales        | DaystoShipAct |
|-------|-------------|--------------|--------------|-------------|--------------|---------------|
| count | 9994.000000 | 9994.000000  | 9994.000000  | 9994.000000 | 9994.000000  | 9994.000000   |
| mean  | 0.156203    | 55190.379428 | 28.651891    | 3.789574    | 229.873324   | 3.9588        |
| std   | 0.206452    | 32063.693350 | 234.255712   | 2.225110    | 623.248946   | 1.7470        |
| min   | 0.000000    | 1040.000000  | -6600.000000 | 1.000000    | 0.000000     | 0.0000        |
| 25%   | 0.000000    | 23223.000000 | 2.000000     | 2.000000    | 17.000000    | 3.0000        |
| 50%   | 0.200000    | 56430.500000 | 9.000000     | 3.000000    | 54.500000    | 4.0000        |
| 75%   | 0.200000    | 90008.000000 | 29.000000    | 5.000000    | 210.000000   | 5.0000        |
| max   | 0.800000    | 99301.000000 | 8400.000000  | 14.000000   | 22638.000000 | 7.0000        |

```
In [5]: df.columns #showing all columns
```

Out[5]: Index(['OrderDate', 'Category', 'City', 'Country', 'CustomerName', 'Discount', 'OrderID', 'PostalCode', 'ProductName', 'Profit', 'Quantity', 'Region', 'Sales', 'Segment', 'ShipDate', 'ShipMode', 'State', 'Sub\_Category', 'DaystoShipActual', 'SalesForecast', 'ShipStatus', 'DaystoShipScheduled', 'OrderProfitable', 'SalesperCustomer', 'ProfitRatio', 'SalesaboveTarget', 'latitude', 'longitude'], dtype='object')

```
In [6]: df.dtypes
```

```
Out[6]: OrderDate          object
        Category          object
        City              object
        Country           object
        CustomerName       object
        Discount           float64
        OrderID            object
        PostalCode         int64
        ProductName        object
        Profit             int64
        Quantity           int64
        Region             object
        Sales              int64
        Segment            object
        ShipDate            object
        ShipMode            object
        State              object
        Sub_Category       object
        DaystoShipActual    int64
        SalesForecast       int64
        ShipStatus         object
        DaystoShipScheduled int64
        OrderProfitable     float64
        SalesperCustomer    float64
        ProfitRatio         float64
        SalesaboveTarget    float64
        latitude            float64
        longitude           float64
        dtype: object
```

According to above data type of 'OrderDate' and 'ShipDate' should convert to DateTime data type

## Data Understanding Summary:

The dataset contains 9994 records with 28 columns, representing sales and customer-related information. The dataset includes the following types of columns:

- **Categorical (Object Type):**
  - Includes fields like `OrderDate`, `Category`, `City`, `Country`, `CustomerName`, `ProductName`, `Region`, `Segment`, `ShipDate`, `ShipMode`, `State`, `Sub_Category`, and `ShipStatus`. These columns provide descriptive information about the sales transactions and customer details.
- **Numerical (Integer and Float Types):**
  - Columns like `Discount`, `Profit`, `Quantity`, `Sales`, `DaystoShipActual`, `SalesForecast`, `DaystoShipScheduled`, `SalesperCustomer`, `ProfitRatio`, `SalesaboveTarget`, `latitude`, and `longitude` contain numerical data, which can be used for analysis such as profit calculations, sales performance, and geographic data.
- **Missing Values:**
  - `OrderProfitable` and `SalesaboveTarget` have missing (NaN) values, indicating that these columns might need further investigation or imputation to handle the missing data.

## Initial Data Quality Report:

- **Total Entries:** 9994 rows.
- **Non-Null Data:** Most columns have 9994 non-null values, except for the `OrderProfitable` and `SalesaboveTarget` columns.
- **Data Types:**
  - `object` : 14 columns (categorical data).
  - `int64` : 7 columns (integer data).
  - `float64` : 7 columns (floating-point numerical data).
- **Missing Data:**
  - `OrderProfitable` and `SalesaboveTarget` contain only missing values (NaN), which should be addressed based on business needs.
- **Memory Usage:** The dataset occupies 2.1 MB of memory.

This dataset is well-structured for analysis but requires handling of missing values and potential conversions (e.g., dates) before deeper analysis or model building.

## Step 2

Data cleaning, preprocessing, and feature engineering.

Deliverables:

- Cleaned dataset
- Python script or notebook

## TRANSFORM of ETL

# Data cleaning and preprocessing

## 1. Handling missing values

```
In [7]: df.isnull().sum() #checking for null values
```

```
Out[7]: OrderDate          0
        Category          0
        City             0
        Country          0
        CustomerName     0
        Discount         0
        OrderID          0
        PostalCode       0
        ProductName      0
        Profit           0
        Quantity         0
        Region           0
        Sales            0
        Segment          0
        ShipDate         0
        ShipMode         0
        State            0
        Sub_Category     0
        DaystoShipActual  0
        SalesForecast     0
        ShipStatus       0
        DaystoShipScheduled 0
        OrderProfitable   9994
        SalesperCustomer  0
        ProfitRatio       0
        SalesaboveTarget  9994
        latitude         0
        longitude        0
        dtype: int64
```

'OrderProfitable' and 'SalesaboveTarget' have null values and should remove these columns

```
In [8]: df = df.drop(['OrderProfitable', 'SalesaboveTarget'], axis=1 )
df.head(5)
```

Out[8]:

|   | OrderDate                | Category        | City         | Country       | CustomerName  | Discount | OrderID        | P |
|---|--------------------------|-----------------|--------------|---------------|---------------|----------|----------------|---|
| 0 | 2011-01-04T00:00:00.000Z | Office Supplies | Houston      | United States | Darren Powers | 0.2      | CA-2011-103800 |   |
| 1 | 2011-01-05T00:00:00.000Z | Office Supplies | Naperville   | United States | Phillina Ober | 0.2      | CA-2011-112326 |   |
| 2 | 2011-01-05T00:00:00.000Z | Office Supplies | Naperville   | United States | Phillina Ober | 0.8      | CA-2011-112326 |   |
| 3 | 2011-01-05T00:00:00.000Z | Office Supplies | Naperville   | United States | Phillina Ober | 0.2      | CA-2011-112326 |   |
| 4 | 2011-01-06T00:00:00.000Z | Office Supplies | Philadelphia | United States | Mick Brown    | 0.2      | CA-2011-141817 |   |

5 rows × 26 columns



```
In [9]: df.isnull().sum() #after removing check again for nulls
```

```
Out[9]: OrderDate          0
        Category          0
        City             0
        Country          0
        CustomerName      0
        Discount          0
        OrderID           0
        PostalCode        0
        ProductName       0
        Profit            0
        Quantity          0
        Region            0
        Sales             0
        Segment           0
        ShipDate          0
        ShipMode           0
        State             0
        Sub_Category      0
        DaystoShipActual   0
        SalesForecast      0
        ShipStatus        0
        DaystoShipScheduled 0
        SalesperCustomer   0
        ProfitRatio       0
        latitude          0
        longitude         0
        dtype: int64
```

## 1. Removing duplicate values

```
In [10]: df.duplicated().sum() #check for duplicate values
```

```
Out[10]: 1
```

```
In [11]: df = df.drop_duplicates() #drop duplicates
```

```
In [12]: df.duplicated().sum() #after dropping check again for duplicate values
```

```
Out[12]: 0
```

## 1. Set up data types for appropriate columns

```
In [13]: df.dtypes #check data types
```

```
Out[13]: OrderDate          object
Category          object
City              object
Country           object
CustomerName      object
Discount          float64
OrderID           object
PostalCode        int64
ProductName       object
Profit            int64
Quantity          int64
Region            object
Sales             int64
Segment           object
ShipDate          object
ShipMode          object
State             object
Sub_Category      object
DaystoShipActual  int64
SalesForecast     int64
ShipStatus        object
DaystoShipScheduled int64
SalesperCustomer  float64
ProfitRatio       float64
latitude          float64
longitude         float64
dtype: object
```

'OrderDate' and 'ShipDate' has 'object' data type should convert into 'datetime'

```
In [14]: df['OrderDate'] = pd.to_datetime(df['OrderDate']) #convert into 'datetime' data type
df['ShipDate'] = pd.to_datetime(df['ShipDate'])
df.dtypes
```

```
Out[14]: OrderDate          datetime64[ns, UTC]
Category          object
City              object
Country           object
CustomerName      object
Discount          float64
OrderID           object
PostalCode        int64
ProductName       object
Profit            int64
Quantity          int64
Region            object
Sales             int64
Segment           object
ShipDate          datetime64[ns, UTC]
ShipMode          object
State             object
Sub_Category      object
DaystoShipActual  int64
SalesForecast     int64
ShipStatus        object
DaystoShipScheduled int64
SalesperCustomer  float64
ProfitRatio       float64
latitude          float64
longitude         float64
dtype: object
```

```
In [15]: df.head(5)
```

```
Out[15]:
```

|   | OrderDate                    | Category           | City         | Country          | CustomerName  | Discount | OrderID                | Pos |
|---|------------------------------|--------------------|--------------|------------------|---------------|----------|------------------------|-----|
| 0 | 2011-01-04<br>00:00:00+00:00 | Office<br>Supplies | Houston      | United<br>States | Darren Powers | 0.2      | CA-<br>2011-<br>103800 |     |
| 1 | 2011-01-05<br>00:00:00+00:00 | Office<br>Supplies | Naperville   | United<br>States | Phillina Ober | 0.2      | CA-<br>2011-<br>112326 |     |
| 2 | 2011-01-05<br>00:00:00+00:00 | Office<br>Supplies | Naperville   | United<br>States | Phillina Ober | 0.8      | CA-<br>2011-<br>112326 |     |
| 3 | 2011-01-05<br>00:00:00+00:00 | Office<br>Supplies | Naperville   | United<br>States | Phillina Ober | 0.2      | CA-<br>2011-<br>112326 |     |
| 4 | 2011-01-06<br>00:00:00+00:00 | Office<br>Supplies | Philadelphia | United<br>States | Mick Brown    | 0.2      | CA-<br>2011-<br>141817 |     |

5 rows × 26 columns

```
In [16]: date_columns = ['OrderDate', 'ShipDate'] #change date format and remove unnecessary time detail in each date

for col in date_columns:
    df[col] = df[col].dt.strftime('%Y-%m-%d')
```

```
In [17]: df.head(5).T
```

Out[17]:

|                            | 0                                                 | 1               | 2                                          | 3                             | 4                                                 |
|----------------------------|---------------------------------------------------|-----------------|--------------------------------------------|-------------------------------|---------------------------------------------------|
| <b>OrderDate</b>           | 2011-01-04                                        | 2011-01-05      | 2011-01-05                                 | 2011-01-05                    | 2011-01-06                                        |
| <b>Category</b>            | Office Supplies                                   | Office Supplies | Office Supplies                            | Office Supplies               | Office Supplies                                   |
| <b>City</b>                | Houston                                           | Naperville      | Naperville                                 | Naperville                    | Philadelphia                                      |
| <b>Country</b>             | United States                                     | United States   | United States                              | United States                 | United States                                     |
| <b>CustomerName</b>        | Darren Powers                                     | Phillina Ober   | Phillina Ober                              | Phillina Ober                 | Mick Brown                                        |
| <b>Discount</b>            | 0.2                                               | 0.2             | 0.8                                        | 0.2                           | 0.2                                               |
| <b>OrderID</b>             | CA-2011-103800                                    | CA-2011-112326  | CA-2011-112326                             | CA-2011-112326                | CA-2011-141817                                    |
| <b>PostalCode</b>          | 77095                                             | 60540           | 60540                                      | 60540                         | 19143                                             |
| <b>ProductName</b>         | Message Book, Wirebound, Four 5 1/2 X 4 Forms/... | Avery 508       | GBC Standard Plastic Binding Systems Combs | SAFCO Boltless Steel Shelving | Avery Hi-Liter EverBold Pen Style Fluorescent ... |
| <b>Profit</b>              | 6                                                 | 4               | -5                                         | -65                           | 5                                                 |
| <b>Quantity</b>            | 2                                                 | 3               | 2                                          | 3                             | 3                                                 |
| <b>Region</b>              | Central                                           | Central         | Central                                    | Central                       | East                                              |
| <b>Sales</b>               | 16                                                | 12              | 4                                          | 273                           | 20                                                |
| <b>Segment</b>             | Consumer                                          | Home Office     | Home Office                                | Home Office                   | Consumer                                          |
| <b>ShipDate</b>            | 2011-01-08                                        | 2011-01-09      | 2011-01-09                                 | 2011-01-09                    | 2011-01-13                                        |
| <b>ShipMode</b>            | Standard Class                                    | Standard Class  | Standard Class                             | Standard Class                | Standard Class                                    |
| <b>State</b>               | Texas                                             | Illinois        | Illinois                                   | Illinois                      | Pennsylvania                                      |
| <b>Sub_Category</b>        | Paper                                             | Labels          | Binders                                    | Storage                       | Art                                               |
| <b>DaystoShipActual</b>    | 4                                                 | 4               | 4                                          | 4                             | 7                                                 |
| <b>SalesForecast</b>       | 22                                                | 15              | 5                                          | 357                           | 26                                                |
| <b>ShipStatus</b>          | Shipped Early                                     | Shipped Early   | Shipped Early                              | Shipped Early                 | Shipped Late                                      |
| <b>DaystoShipScheduled</b> | 6                                                 | 6               | 6                                          | 6                             | 6                                                 |
| <b>SalesperCustomer</b>    | 16.45                                             | 11.78           | 3.54                                       | 272.74                        | 19.54                                             |
| <b>ProfitRatio</b>         | 33.8                                              | 36.3            | -155.0                                     | -23.8                         | 25.0                                              |
| <b>latitude</b>            | 29.8941                                           | 41.7662         | 41.7662                                    | 41.7662                       | 39.9448                                           |
| <b>longitude</b>           | -95.6481                                          | -88.141         | -88.141                                    | -88.141                       | -75.2288                                          |

```
In [18]: df['OrderDate'] = pd.to_datetime(df['OrderDate']) #convert into 'datetime' data type
df['ShipDate'] = pd.to_datetime(df['ShipDate'])
df.dtypes
```

```
Out[18]: OrderDate          datetime64[ns]
Category          object
City              object
Country           object
CustomerName      object
Discount          float64
OrderID           object
PostalCode        int64
ProductName       object
Profit            int64
Quantity          int64
Region            object
Sales             int64
Segment           object
ShipDate          datetime64[ns]
ShipMode          object
State             object
Sub_Category      object
DaystoShipActual  int64
SalesForecast     int64
ShipStatus        object
DaystoShipScheduled int64
SalesperCustomer  float64
ProfitRatio       float64
latitude          float64
longitude         float64
dtype: object
```

```
In [19]: df.head(5)
```

```
Out[19]:
```

|   | OrderDate  | Category        | City         | Country       | CustomerName  | Discount | OrderID        | PostalC |
|---|------------|-----------------|--------------|---------------|---------------|----------|----------------|---------|
| 0 | 2011-01-04 | Office Supplies | Houston      | United States | Darren Powers | 0.2      | CA-2011-103800 | 77      |
| 1 | 2011-01-05 | Office Supplies | Naperville   | United States | Phillina Ober | 0.2      | CA-2011-112326 | 60      |
| 2 | 2011-01-05 | Office Supplies | Naperville   | United States | Phillina Ober | 0.8      | CA-2011-112326 | 60      |
| 3 | 2011-01-05 | Office Supplies | Naperville   | United States | Phillina Ober | 0.2      | CA-2011-112326 | 60      |
| 4 | 2011-01-06 | Office Supplies | Philadelphia | United States | Mick Brown    | 0.2      | CA-2011-141817 | 19      |

5 rows × 26 columns

## 1. Label encoding categorical data

### 4.1 label encoding for 'Category' column

```
In [20]: df['Category'].value_counts() #counting values for unique category
```

```
Out[20]: Category
Office Supplies    6026
Furniture          2120
Technology         1847
Name: count, dtype: int64
```

```
In [21]: # Custom mapping
category_map = {
    'Office Supplies': 1,
    'Furniture': 2,
    'Technology': 3
}

df['Category_encoded'] = df['Category'].map(category_map)
```

### 4.2 label encoding for 'Region' column

```
In [22]: df['Region'].value_counts() #counting values for unique Region
```

```
Out[22]: Region
West      3203
East      2847
Central   2323
South     1620
Name: count, dtype: int64
```

```
In [23]: # Custom mapping
category_map = {
    'West': 1,
    'East': 2,
    'Central': 3,
    'South': 4
}

df['Region_encoded'] = df['Region'].map(category_map)
```

#### 4.3 Label encoding for 'Segment' column

```
In [24]: df['Segment'].value_counts() #counting values for unique segment
```

```
Out[24]: Segment
Consumer      5191
Corporate     3020
Home Office   1782
Name: count, dtype: int64
```

```
In [25]: category_map = {
    'Consumer': 1,
    'Corporate': 2,
    'Home Office': 3
}

df['Segment_encoded'] = df['Segment'].map(category_map)
```

#### 4.4 Label encoding for 'ShipMode' column

```
In [26]: df['ShipMode'].value_counts() #counting values for unique ShipMode
```

```
Out[26]: ShipMode
Standard Class    5967
Second Class      1945
First Class       1538
Same Day          543
Name: count, dtype: int64
```



```
In [27]: category_map = {
          'Standard Class': 1,
          'Second Class': 2,
          'First Class': 3,
          'Same Day': 4
        }

df['ShipMode_encoded'] = df['ShipMode'].map(category_map)
```

#### 4.5 Label encoding for 'ShipStatus' column

```
In [28]: df['ShipStatus'].value_counts() #counting values for unique ShipSta
tus
```

```
Out[28]: ShipStatus
Shipped Early      4907
Shipped Late       2639
Shipped On Time    2447
Name: count, dtype: int64
```

```
In [29]: category_map = {
          'Shipped Early': 1,
          'Shipped Late': 2,
          'Shipped On Time': 3
        }

df['ShipStatus_encoded'] = df['ShipStatus'].map(category_map)
```

#### 4.6 Label encoding for 'Sub\_Category' column

```
In [30]: df['Sub_Category'].value_counts()
```

```
Out[30]: Sub_Category
Binders      1523
Paper        1370
Furnishings   957
Phones       889
Storage      846
Art          796
Accessories  775
Chairs       616
Appliances   466
Labels       364
Tables       319
Envelopes    254
Bookcases    228
Fasteners    217
Supplies     190
Machines     115
Copiers       68
Name: count, dtype: int64
```

```
In [31]: category_map = {
    'Binders': 1,
    'Paper': 2,
    'Furnishings': 3,
    'Phones' : 4,
    'Storage':5,
    'Art':6,
    'Accessories':7,
    'Chairs':8,
    'Appliances':9,
    'Labels':10,
    'Tables':11,
    'Envelopes':12,
    'Bookcases':13,
    'Fasteners':14,
    'Supplies':15,
    'Machines':16,
    'Copiers':17
}

df['Sub_Category_encoded'] = df['Sub_Category'].map(category_map)
```

```
In [32]: df.head(5) #after encoding check dataset
```

Out[32]:

|   | OrderDate  | Category        | City         | Country       | CustomerName  | Discount | OrderID        | PostalC |
|---|------------|-----------------|--------------|---------------|---------------|----------|----------------|---------|
| 0 | 2011-01-04 | Office Supplies | Houston      | United States | Darren Powers | 0.2      | CA-2011-103800 | 77      |
| 1 | 2011-01-05 | Office Supplies | Naperville   | United States | Phillina Ober | 0.2      | CA-2011-112326 | 60      |
| 2 | 2011-01-05 | Office Supplies | Naperville   | United States | Phillina Ober | 0.8      | CA-2011-112326 | 60      |
| 3 | 2011-01-05 | Office Supplies | Naperville   | United States | Phillina Ober | 0.2      | CA-2011-112326 | 60      |
| 4 | 2011-01-06 | Office Supplies | Philadelphia | United States | Mick Brown    | 0.2      | CA-2011-141817 | 19      |

5 rows × 32 columns

## Feature engineering

```
In [33]: df.dtypes #check again for data types
```

```
Out[33]: OrderDate          datetime64[ns]
Category          object
City              object
Country           object
CustomerName      object
Discount          float64
OrderID           object
PostalCode        int64
ProductName       object
Profit            int64
Quantity          int64
Region            object
Sales             int64
Segment           object
ShipDate          datetime64[ns]
ShipMode          object
State             object
Sub_Category      object
DaystoShipActual  int64
SalesForecast     int64
ShipStatus        object
DaystoShipScheduled int64
SalesperCustomer  float64
ProfitRatio       float64
latitude          float64
longitude         float64
Category_encoded  int64
Region_encoded    int64
Segment_encoded   int64
ShipMode_encoded  int64
ShipStatus_encoded int64
Sub_Category_encoded int64
dtype: object
```

Drop unnecassary columns

```
In [34]: drop_columns = ['Category', 'Country', 'CustomerName', 'Region', 'Segment', 'ShipMode', 'Sub_Category', 'ShipStatus', ]
df_v1 = df.drop(drop_columns, axis=1)
df_v1.head(10)
```

Out[34]:

|   | OrderDate  | City         | Discount | OrderID        | PostalCode | ProductName                                       | Profit | Quantity |
|---|------------|--------------|----------|----------------|------------|---------------------------------------------------|--------|----------|
| 0 | 2011-01-04 | Houston      | 0.2      | CA-2011-103800 | 77095      | Message Book, Wirebound, Four 5 1/2 X 4 Forms/... | 6      | 2        |
| 1 | 2011-01-05 | Naperville   | 0.2      | CA-2011-112326 | 60540      | Avery 508                                         | 4      | 3        |
| 2 | 2011-01-05 | Naperville   | 0.8      | CA-2011-112326 | 60540      | GBC Standard Plastic Binding Systems Combs        | -5     | 2        |
| 3 | 2011-01-05 | Naperville   | 0.2      | CA-2011-112326 | 60540      | SAFCO Boltless Steel Shelving                     | -65    | 3        |
| 4 | 2011-01-06 | Philadelphia | 0.2      | CA-2011-141817 | 19143      | Avery Hi-Liter EverBold Pen Style Fluorescent ... | 5      | 3        |
| 5 | 2011-01-07 | Henderson    | 0.0      | CA-2011-167199 | 42420      | Global Deluxe High-Back Managers Chair            | 746    | 9        |
| 6 | 2011-01-07 | Athens       | 0.0      | CA-2011-106054 | 30605      | Dixon Prang Watercolor Pencils, 10-Color Set w... | 5      | 3        |
| 7 | 2011-01-07 | Henderson    | 0.0      | CA-2011-167199 | 42420      | Alliance Super-Size Bands, Assorted Sizes         | 0      | 4        |
| 8 | 2011-01-07 | Henderson    | 0.0      | CA-2011-167199 | 42420      | Ibico Hi-Tech Manual Binding System               | 274    | 2        |
| 9 | 2011-01-07 | Henderson    | 0.0      | CA-2011-167199 | 42420      | Rogers Handheld Barrel Pencil Sharpener           | 1      | 2        |

10 rows × 24 columns

In [35]: df\_v1.shape *#reduced fetures from 28 into 24*

Out[35]: (9993, 24)

```
In [36]: df_v1.dtypes
```

```
Out[36]: OrderDate          datetime64[ns]
City                object
Discount            float64
OrderID             object
PostalCode          int64
ProductName          object
Profit              int64
Quantity            int64
Sales               int64
ShipDate            datetime64[ns]
State               object
DaystoShipActual    int64
SalesForecast       int64
DaystoShipScheduled int64
SalesperCustomer    float64
ProfitRatio         float64
latitude            float64
longitude            float64
Category_encoded    int64
Region_encoded      int64
Segment_encoded     int64
ShipMode_encoded    int64
ShipStatus_encoded  int64
Sub_Category_encoded int64
dtype: object
```

## Step 3

Schema Design.

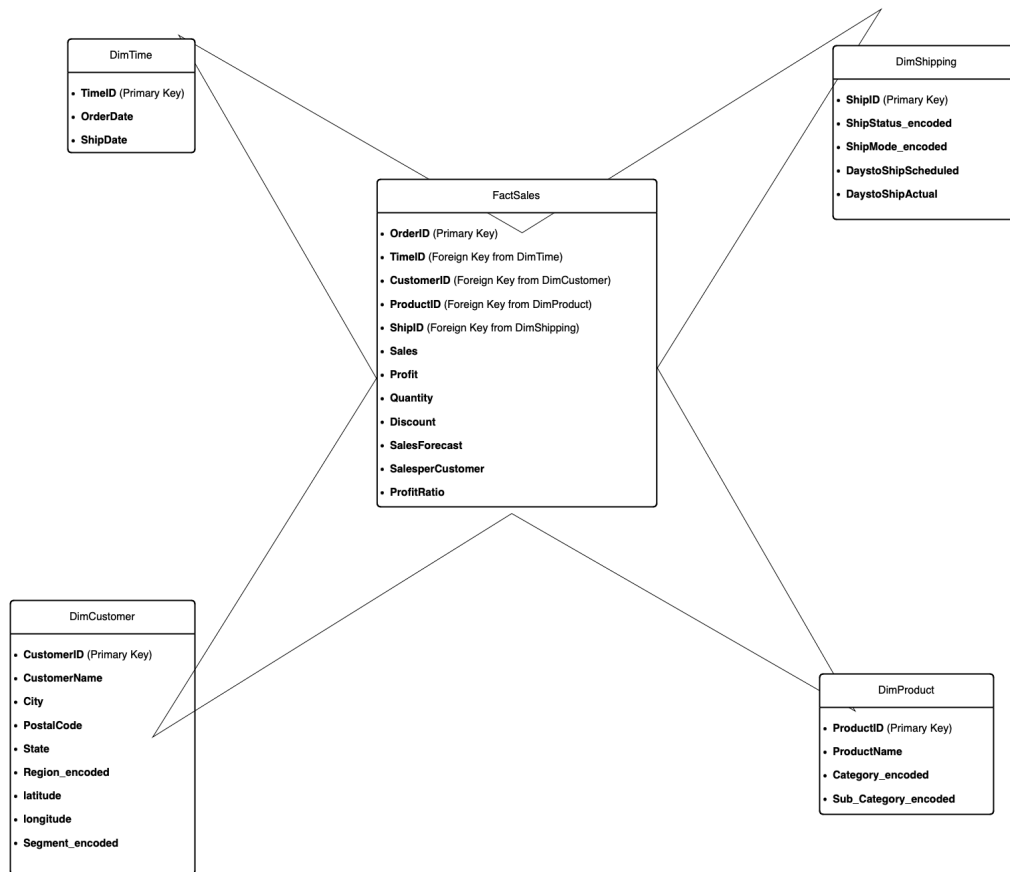
Deliverables:

- Justification for schema choice
- Schema Diagram
- ER Diagram showing table relationships

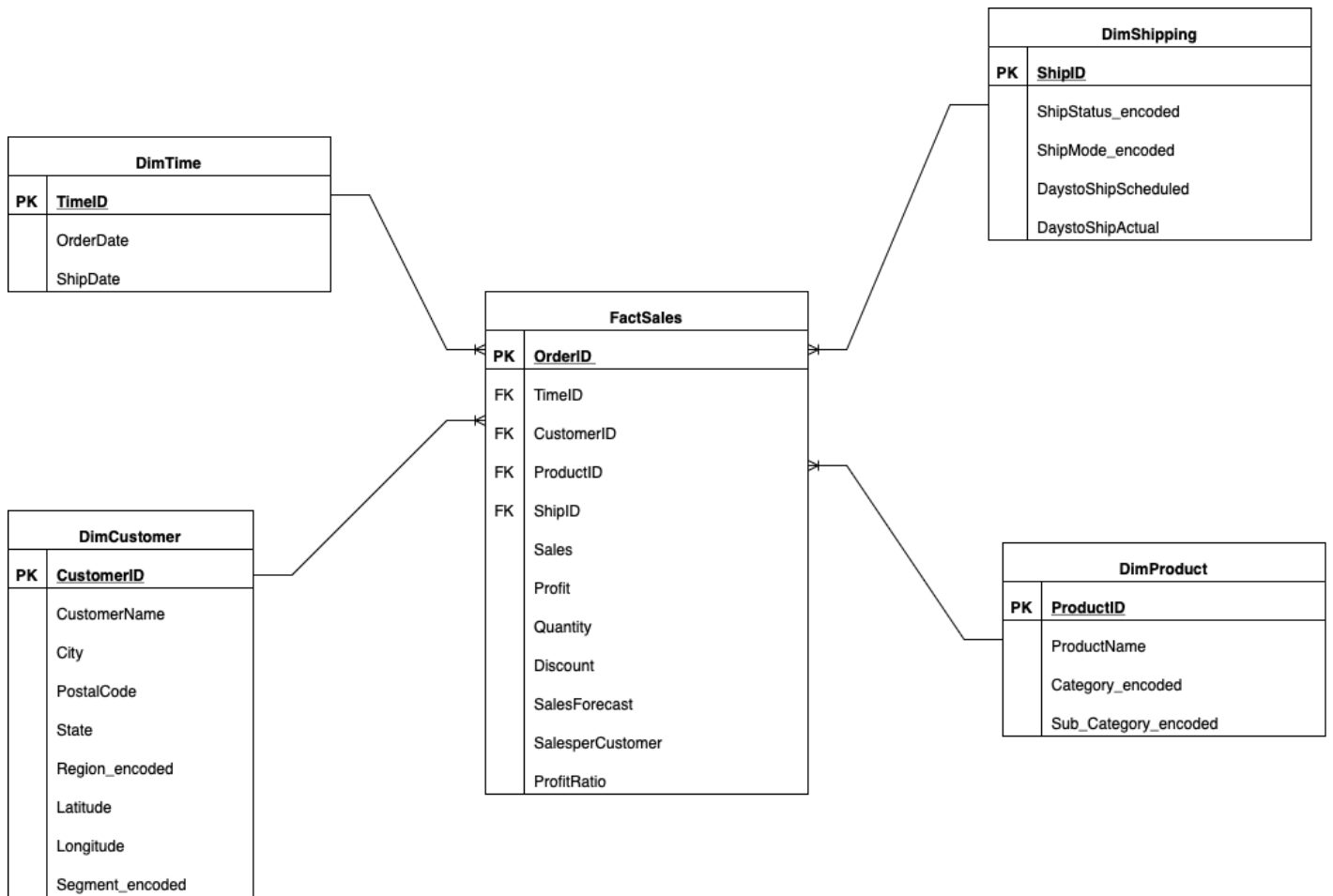
### 1. Justification for select star schema

The Star Schema is a great choice for your project because it offers a simple and clear structure, with the Fact Table (e.g., FactSales) at the center and Dimension Tables (e.g., DimTime, DimCustomer, DimProduct, DimShipping) around it. This design makes querying easy and fast, especially for business analysis like sales by customer or product. It helps to minimize data redundancy and improves query performance. The schema also makes time-based analysis (like sales trends) easier and can scale as your business grows, allowing you to add new dimensions without disrupting the existing structure. Overall, it's efficient, flexible, and perfect for generating reports and insights.

### 1. Star schema design



3.ER diagram



## Step 4

Implementing the in Snowflake (or Similar Warehouse).

Create tables in Snowflake according to the schema design and load cleaned data into the appropriate tables.

Deliverables:

- SQL scripts for schema creation
- Screenshots or logs showing successful data load

**Load cleaned data into STAGE area of snowflake schema**

Q Search

ASSIGNMENT\_DB

ASSIGNMENT\_SCHEMA

Tables

DIMCUSTOMER

DIMPRODUCT

DIMSHIPPING

DIMTIME

FACTSALES

Stages

STG\_SALES

INFORMATION\_SCHEMA

PUBLIC

ASSIGNMENT\_DB / ASSIGNMENT\_SCHEMA / STG\_SALES

Internal Stage ACCOUNTADMIN 4 minutes ago

Stage FilesStage DetailsLineage

STG\_SALES (1 File)

Q SearchPROJECT\_WH

| NAME                   | SIZE  | LAST MODIFIED  |  |
|------------------------|-------|----------------|--|
| cleaned_sales_data.csv | 1.5MB | 57 seconds ago |  |

## Create star schema in snowflake using SQL scripts

### 1. SQL script for FACTSALES

Q Search

ASSIGNMENT\_DB

ASSIGNMENT\_SCHEMA

Tables

DIMCUSTOMER

DIMPRODUCT

DIMSHIPPING

DIMTIME

FACTSALES

Stages

INFORMATION\_SCHEMA

PUBLIC

DBT\_DB

MYDB

PROJECT\_DB

SNOWFLAKE

SNOWFLAKE\_SAMPLE\_DATA

ASSIGNMENT\_DB / ASSIGNMENT\_SCHEMA / FACTSALES

Table ACCOUNTADMIN 21 minutes ago 0 0.0B

Table DetailsColumnsData PreviewCopy HistoryLineage

Table definition

```
1 create or replace TABLE ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.FACTSALES (  
2     ORDERID VARCHAR(90) NOT NULL,  
3     TIMEID VARCHAR(90),  
4     CUSTOMERID VARCHAR(90),  
5     PRODUCTID VARCHAR(90),  
6     SHIPID VARCHAR(90),  
7     SALES NUMBER(10,2),  
8     PROFIT NUMBER(10,2),  
9     QUANTITY NUMBER(38,0),  
10    DISCOUNT NUMBER(5,2),  
11    SALESFORECAST NUMBER(10,2),  
12    SALESPERCUSTOMER NUMBER(10,2),  
13    PROFITRATIO NUMBER(5,2),  
14    primary key (ORDERID),  
15    foreign key (TIMEID) references  
    ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMTIME(TIMEID),  
16    foreign key (CUSTOMERID) references  
    ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMCUSTOMER(CUSTOMERID),  
17    foreign key (PRODUCTID) references  
    ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMPRODUCT(PRODUCTID),  
18    foreign key (SHIPID) references  
    ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMSHIPPING(SHIPID)  
19 );
```



1. SQL script for DIMTIME

Q Search

ASSIGNMENT\_DB

ASSIGNMENT\_SCHEMA

Tables

DIMCUSTOMER

DIMPRODUCT

DIMSHIPPING

**DIMTIME**

FACTSALES

INFORMATION\_SCHEMA

PUBLIC

DBT\_DB

MYDB

PROJECT\_DB

SNOWFLAKE

SNOWFLAKE\_SAMPLE\_DATA

ASSIGNMENT\_DB / ASSIGNMENT\_SCHEMA / DIMTIME

Table ACCOUNTADMIN 2 minutes ago 0 0.0B

Table Details Columns Data Preview Copy History Lineage

Write a description...

Table definition

```
1 create or replace TABLE ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMTIME (  
2     TIMEID NUMBER(38,0) NOT NULL,  
3     ORDERDATE DATE,  
4     SHIPDATE DATE,  
5     primary key (TIMEID)  
6 );
```

1. SQL script for DIMSHIPPING

Q Search

ASSIGNMENT\_DB

ASSIGNMENT\_SCHEMA

Tables

DIMCUSTOMER

DIMPRODUCT

**DIMSHIPPING**

DIMTIME

FACTSALES

INFORMATION\_SCHEMA

PUBLIC

DBT\_DB

MYDB

PROJECT\_DB

SNOWFLAKE

SNOWFLAKE\_SAMPLE\_DATA

ASSIGNMENT\_DB / ASSIGNMENT\_SCHEMA / DIMSHIPPING

Table ACCOUNTADMIN 2 minutes ago 0 0.0B

Table Details Columns Data Preview Copy History Lineage

Write a description...

Table definition

```
1 create or replace TABLE ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMSHIPPING (  
2     SHIPID NUMBER(38,0) NOT NULL,  
3     SHIPSTATUS_ENCODED VARCHAR(255),  
4     SHIPMODE_ENCODED VARCHAR(255),  
5     DAYSTOSHIPSCHEDULED NUMBER(38,0),  
6     DAYSTOSHIPACTUAL NUMBER(38,0),  
7     primary key (SHIPID)  
8 );
```

## 1. SQL script for DIMPRODUCT

Search

ASSIGNMENT\_DB

ASSIGNMENT\_SCHEMA

Tables

DIMCUSTOMER

**DIMPRODUCT**

DIMSHIPPING

DIMTIME

FACTSALES

INFORMATION\_SCHEMA

PUBLIC

DBT\_DB

MYDB

PROJECT\_DB

SNOWFLAKE

SNOWFLAKE\_SAMPLE\_DATA

ASSIGNMENT\_DB / ASSIGNMENT\_SCHEMA / DIMPRODUCT

Table ACCOUNTADMIN 2 minutes ago 0 0.0B

Table Details Columns Data Preview Copy History Lineage

Write a description...

Table definition

```
1 create or replace TABLE ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMPRODUCT (  
2     PRODUCTID NUMBER(38,0) NOT NULL,  
3     PRODUCTNAME VARCHAR(255),  
4     CATEGORY_ENCODED VARCHAR(255),  
5     SUB_CATEGORY_ENCODED VARCHAR(255),  
6     primary key (PRODUCTID)  
7 );
```

## 1. SQL script for DIMCUSTOMER

Search

ASSIGNMENT\_DB

ASSIGNMENT\_SCHEMA

Tables

**DIMCUSTOMER**

DIMPRODUCT

DIMSHIPPING

DIMTIME

FACTSALES

INFORMATION\_SCHEMA

PUBLIC

DBT\_DB

MYDB

PROJECT\_DB

SNOWFLAKE

SNOWFLAKE\_SAMPLE\_DATA

ASSIGNMENT\_DB / ASSIGNMENT\_SCHEMA / DIMCUSTOMER

Table ACCOUNTADMIN 5 minutes ago 0 0.0B

Table Details Columns Data Preview Copy History Lineage

Table definition

```
1 create or replace TABLE ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMCUSTOMER (  
2     CUSTOMERID NUMBER(38,0) NOT NULL,  
3     CUSTOMERNAME VARCHAR(255),  
4     CITY VARCHAR(255),  
5     POSTALCODE VARCHAR(20),  
6     STATE VARCHAR(255),  
7     REGION_ENCODED VARCHAR(255),  
8     LATITUDE NUMBER(10,6),  
9     LONGITUDE NUMBER(10,6),  
10    SEGMENT_ENCODED VARCHAR(255),  
11    primary key (CUSTOMERID)  
12 );
```

Show less

```
In [37]: # export cleaned csv  
df_v1.to_csv('cleaned_sales_data.csv', index=False)
```

## LOAD of ETL

1. LOAD data into FACTSALES

Q Search

ASSIGNMENT\_DB

ASSIGNMENT\_SCHEMA

Tables

DIMCUSTOMER

DIMPRODUCT

DIMSHIPPING

DIMTIME

FACTSALES

Stages

INFORMATION\_SCHEMA

PUBLIC

DBT\_DB

MYDB

PROJECT\_DB

SNOWFLAKE

SNOWFLAKE\_SAMPLE\_DATA

ASSIGNMENT\_DB / ASSIGNMENT\_SCHEMA / FACTSALES

Table ACCOUNTADMIN 14 minutes ago 10.0K 140.0KB

Table Details Columns Data Preview Copy History Lineage

Last 7 days Status All Pipe All 1 Copy

Search PROJECT\_WH

Loaded (1)

1

0 May 9 May 10 May 11 May 12 May 13 May 14 May 15

FILE NAME LOADED SIZE ROWS STATUS SOUR... PIPE

cleaned\_sales\_data.csv 12m ago 1.5MB 10K Loaded

1. LOAD data into DIMTIME

Q Search

ASSIGNMENT\_DB

ASSIGNMENT\_SCHEMA

Tables

DIMCUSTOMER

DIMPRODUCT

DIMSHIPPING

DIMTIME

FACTSALES

Stages

INFORMATION\_SCHEMA

PUBLIC

DBT\_DB

MYDB

PROJECT\_DB

SNOWFLAKE

SNOWFLAKE\_SAMPLE\_DATA

ASSIGNMENT\_DB / ASSIGNMENT\_SCHEMA / DIMTIME

Table ACCOUNTADMIN 10 minutes ago 10.0K 28.0KB

Table Details Columns Data Preview Copy History Lineage

Last 7 days Status All Pipe All 1 Copy

Search PROJECT\_WH

Loaded (1)

1

0 May 9 May 10 May 11 May 12 May 13 May 14 May 15

FILE NAME LOADED SIZE ROWS STATUS SOUR... PIPE

cleaned\_sales\_data.csv 9m ago 1.5MB 10K Loaded

1. LOAD data into DIMSHIPPING

Q Search

ASSIGNMENT\_DB

ASSIGNMENT\_SCHEMA

Tables

DIMCUSTOMER

DIMPRODUCT

DIMSHIPPING

DIMTIME

FACTSALES

Stages

INFORMATION\_SCHEMA

PUBLIC

DBT\_DB

MYDB

PROJECT\_DB

SNOWFLAKE

SNOWFLAKE\_SAMPLE\_DATA

ASSIGNMENT\_DB / ASSIGNMENT\_SCHEMA / DIMSHIPPING

Table ACCOUNTADMIN 10 minutes ago 10.0K 17.0KB

Table Details Columns Data Preview Copy History Lineage

Last 7 days Status All Pipe All 1 Copy

Search PROJECT\_WH

Loaded (1)

1

0 May 9 May 10 May 11 May 12 May 13 May 14 May 15

FILE NAME LOADED SIZE ROWS STATUS SOUR... PIPE

cleaned\_sales\_data.csv 10m ago 1.5MB 10K Loaded

1. LOAD data into DIMPRODUCT

Q Search

ASSIGNMENT\_DB

ASSIGNMENT\_SCHEMA

Tables

DIMCUSTOMER

DIMPRODUCT

DIMSHIPPING

DIMTIME

FACTSALES

Stages

INFORMATION\_SCHEMA

PUBLIC

DBT\_DB

MYDB

PROJECT\_DB

SNOWFLAKE

SNOWFLAKE\_SAMPLE\_DATA

ASSIGNMENT\_DB / ASSIGNMENT\_SCHEMA / DIMPRODUCT

Table ACCOUNTADMIN 10 minutes ago 10.0K 176.0KB

Table Details Columns Data Preview Copy History Lineage

Last 7 days Status All Pipe All 1 Copy

Search PROJECT\_WH

Loaded (1)

1

0 May 9 May 10 May 11 May 12 May 13 May 14 May 15

FILE NAME LOADED SIZE ROWS STATUS SOUR... PIPE

cleaned\_sales\_data.csv 9m ago 1.5MB 10K Loaded

## 1. LOAD data into DIMCUSTOMER

Q Search

ASSIGNMENT\_DB

ASSIGNMENT\_SCHEMA

Tables

DIMCUSTOMER

DIMPRODUCT

DIMSHIPPING

DIMTIME

FACTSALES

Stages

INFORMATION\_SCHEMA

PUBLIC

DBT\_DB

MYDB

PROJECT\_DB

SNOWFLAKE

SNOWFLAKE\_SAMPLE\_DATA

ASSIGNMENT\_DB / ASSIGNMENT\_SCHEMA / DIMCUSTOMER

Table ACCOUNTADMIN 8 minutes ago 10.0K 117.0KB

Table Details Columns Data Preview Copy History Lineage

Last 7 days Status All Pipe All 1 Copy

Search PROJECT\_WH

Loaded (1)

1

0 May 9 May 10 May 11 May 12 May 13 May 14 May 15

FILE NAME

LOADED

SIZE

ROWS

STATUS

SOUR...

PIPE

cleaned\_sales\_data.csv

7m ago

1.5MB

10K

Loaded

—

## Step 5

Data Mining Tasks (Using Data from Warehouse).

Use necessary datamining techniques to complete below tasks.

- Predict Shipment Status
- Predict Sales Forecast
- Predict Order Profitability
- Cluster Orders by Buying Behavior - Group orders/customers by similar buying/shipping patterns
- Visualize shipment performance across geographies
- Analyze delivery delays or profit trends by latitude/longitude

Configuring snowpark with python in jupyter notebook

```
In [38]: # import snowflake.connector

# conn = snowflake.connector.connect(
#     user='Danika',
#     password='wnn2mmA5BsuQPg5',
#     account='WJB17972',
#     warehouse='COMPUTE_WH',
#     database='SALES',
#     schema='SALES_DATA'
# )
```

```
In [39]: import snowflake.connector

try:
    conn = snowflake.connector.connect(
        user='subhabanda',
        password='snowSubha2000#',
        #account='https://yj85186.ap-south-1.aws.snowflakecomputin
g.com',
        account='POJKKKP-RJ66138',
        warehouse='PROJECT_WH',
        database='ASSIGNMENT_DB',
        schema='ASSIGNMENT_SCHEMA'
    )
    print("Connected to Snowflake successfully.")
except Exception as e:
    print(f"Connection failed: {e}")
```

Connected to Snowflake successfully.

```
In [40]: # from snowflake.snowpark import Session

# connection_parameters = {
#     "account": "POJKKKP-RJ66138",
#     "user": "subhabanda",
#     "password": "snowSubha2000#",
#     "role": "ACCOUNTADMIN",
#     "warehouse": "PROJECT_WH",
#     "database": "ASSIGNMENT_DB",
#     "schema": "ASSIGNMENT_SCHEMA"
# }

# try:
#     session = Session.builder.configs(connection_parameters).crea
te()
#     print("Connected to Snowflake successfully.")
# except Exception as e:
#     print(f"Connection failed: {e}")
```

In [ ]:

```
In [41]: query = """
WITH t1 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERDATE) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMTIME
),
t2 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY SHIPSTATUS_ENCODED) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMSHIPPING
)
SELECT
    t1.ORDERDATE, t1.SHIPDATE,
    t2.SHIPSTATUS_ENCODED, t2.SHIPMODE_ENCODED,
    t2.DAYSTOSHIPSCHEDULED, t2.DAYSTOSHIPACTUAL
FROM
    t1
JOIN
    t2 ON t1.rn = t2.rn;

"""
```

```
In [42]: try:
        df = pd.read_sql(query, conn)
    except Exception as e:
        print("Error:", e)
```

```
/var/folders/4k/wvlp711yz8cxnbyqjw1zlh0000gn/T/ipykernel_38808/25130294.py:2: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
    df = pd.read_sql(query, conn)
```

## 5.1 Predict Shipment Status

### *Pulling data from warehouse*

```
In [43]: query1 = """
WITH t1 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERDATE) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMTIME
),
t2 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY SHIPSTATUS_ENCODED) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMSHIPPING
),
t3 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY PRODUCTNAME) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMPRODUCT
),
t4 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY CITY) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMCUSTOMER
),
t5 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERID) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.FACTSALES
)

SELECT
    t2.SHIPSTATUS_ENCODED,
    t2.SHIPMODE_ENCODED,
    t2.DAYSTOSHIPSCHEDULED,
    t2.DAYSTOSHIPACTUAL,
    t3.PRODUCTNAME,
    t3.CATEGORY_ENCODED,
    t3.SUB_CATEGORY_ENCODED,
    t4.CITY,
    t4.LATITUDE,
    t4.LONGITUDE,
    t4.POSTALCODE,
    t4.STATE,
    t4.REGION_ENCODED,
    t4.SEGMENT_ENCODED,
    t5.DISCOUNT,
    t5.PROFIT,
    t5.PROFITRATIO,
    t5.QUANTITY,
    t5.SALES
FROM
    t5
JOIN t2 ON t5.rn = t2.rn
JOIN t3 ON t5.rn = t3.rn
JOIN t4 ON t5.rn = t4.rn
JOIN t1 ON t5.rn = t1.rn;

"""
```



```
In [44]: df_q1 = pd.read_sql(query1, conn)

/var/folders/4k/wvlp711yz8cxnbyqjw1zlh0000gn/T/ipykernel_38808/2758062646.py:1: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
  df_q1 = pd.read_sql(query1, conn)
```

```
In [45]: df_q1.columns
```

```
Out[45]: Index(['SHIPSTATUS_ENCODED', 'SHIPMODE_ENCODED', 'DAYSTOSHIPSCHEDULED',
               'DAYSTOSHIPACTUAL', 'PRODUCTNAME', 'CATEGORY_ENCODED',
               'SUB_CATEGORY_ENCODED', 'CITY', 'LATITUDE', 'LONGITUDE', 'POSTALCODE',
               'STATE', 'REGION_ENCODED', 'SEGMENT_ENCODED', 'DISCOUNT', 'PROFIT',
               'PROFITRATIO', 'QUANTITY', 'SALES'],
              dtype='object')
```

```
In [46]: df_q1.shape
```

```
Out[46]: (9993, 19)
```

```
In [47]: df_q1.head()
```

```
Out[47]:
```

|   | SHIPSTATUS_ENCODED | SHIPMODE_ENCODED | DAYSTOSHIPSCHEDULED | DAYSTOSHIP/ |
|---|--------------------|------------------|---------------------|-------------|
| 0 | 1                  | 1                | 6                   |             |
| 1 | 1                  | 1                | 6                   |             |
| 2 | 1                  | 1                | 6                   |             |
| 3 | 1                  | 1                | 6                   |             |
| 4 | 1                  | 1                | 6                   |             |

## Feature engineering and preprocessing

in above df\_q1 dataframe has importance categorical data. First we should convert those into numerical using label encoding

```
In [48]: from sklearn.preprocessing import LabelEncoder

le_city = LabelEncoder()
df_q1['CITY_encoded'] = le_city.fit_transform(df_q1['CITY'])

le_productname = LabelEncoder()
df_q1['PRODUCTNAME_encoded'] = le_city.fit_transform(df_q1['PRODUCTNAME'])

le_state = LabelEncoder()
df_q1['STATE_encoded'] = le_city.fit_transform(df_q1['STATE'])
```

```
In [49]: df_q1.head()
```

Out[49]:

|   | SHIPSTATUS_ENCODED | SHIPMODE_ENCODED | DAYSTOSHIPSCHEDULED | DAYSTOSHIP/ |
|---|--------------------|------------------|---------------------|-------------|
| 0 | 1                  | 1                | 6                   |             |
| 1 | 1                  | 1                | 6                   |             |
| 2 | 1                  | 1                | 6                   |             |
| 3 | 1                  | 1                | 6                   |             |
| 4 | 1                  | 1                | 6                   |             |

5 rows × 22 columns

```
In [50]: #drop unnecassary categorical columns
df_q1.drop(['PRODUCTNAME', 'CITY', 'STATE'], axis=1, inplace=True)
```

```
In [51]: df_q1.head()
```

```
Out[51]:
```

|   | SHIPSTATUS_ENCODED | SHIPMODE_ENCODED | DAYSTOSHIPSCHEDULED | DAYSTOSHIP/ |
|---|--------------------|------------------|---------------------|-------------|
| 0 | 1                  | 1                | 6                   |             |
| 1 | 1                  | 1                | 6                   |             |
| 2 | 1                  | 1                | 6                   |             |
| 3 | 1                  | 1                | 6                   |             |
| 4 | 1                  | 1                | 6                   |             |

```
In [52]: df_q1.shape
```

```
Out[52]: (9993, 19)
```

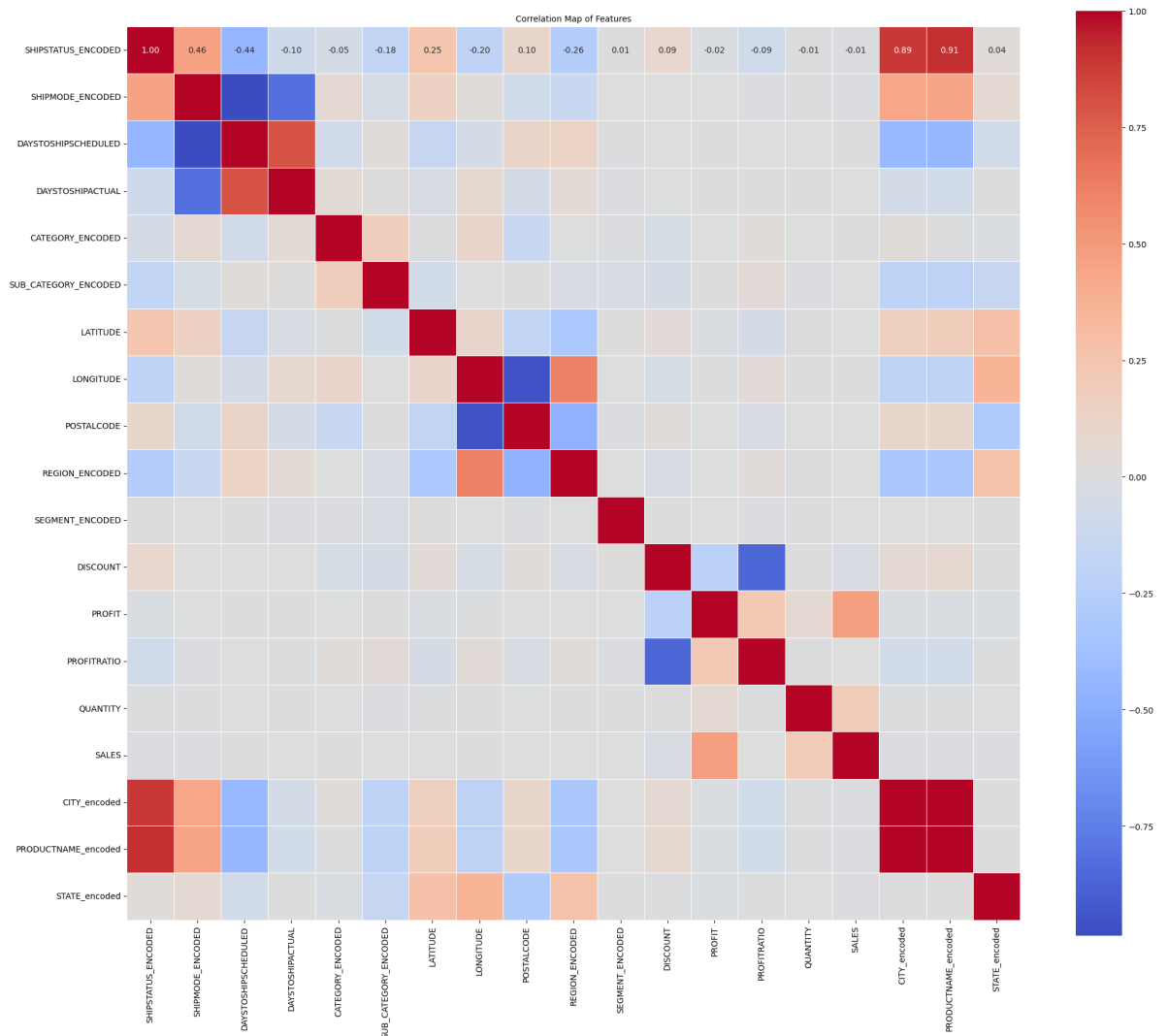
```
In [53]: df_q1_copy = df_q1.copy() #make copy from original
```

### **Data mining techniques, evaluation**

```
In [54]: import matplotlib.pyplot as plt
import seaborn as sns

corr_matrix = df_q1.corr()

plt.figure(figsize=(24, 20))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm', square=True, linewidths=0.5)
plt.title("Correlation Map of Features", fontsize=10)
plt.show()
```



```
In [55]: Y = df_q1_copy['SHIPSTATUS_ENCODED'] #CREATE target variable
X = df_q1_copy.drop(['SHIPSTATUS_ENCODED'], axis=1) #create features
```

```
In [56]: from sklearn.model_selection import train_test_split

#splitting dataset into train and test(20%)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
```

```
In [57]: from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, Y_train)
```

```
Out[57]: ▼ RandomForestClassifier
RandomForestClassifier(random_state=42)
```

<https://scikit-learn.org/1.6/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

**Evaluation with accuracy**

```
In [58]: y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(Y_test, y_pred))
```

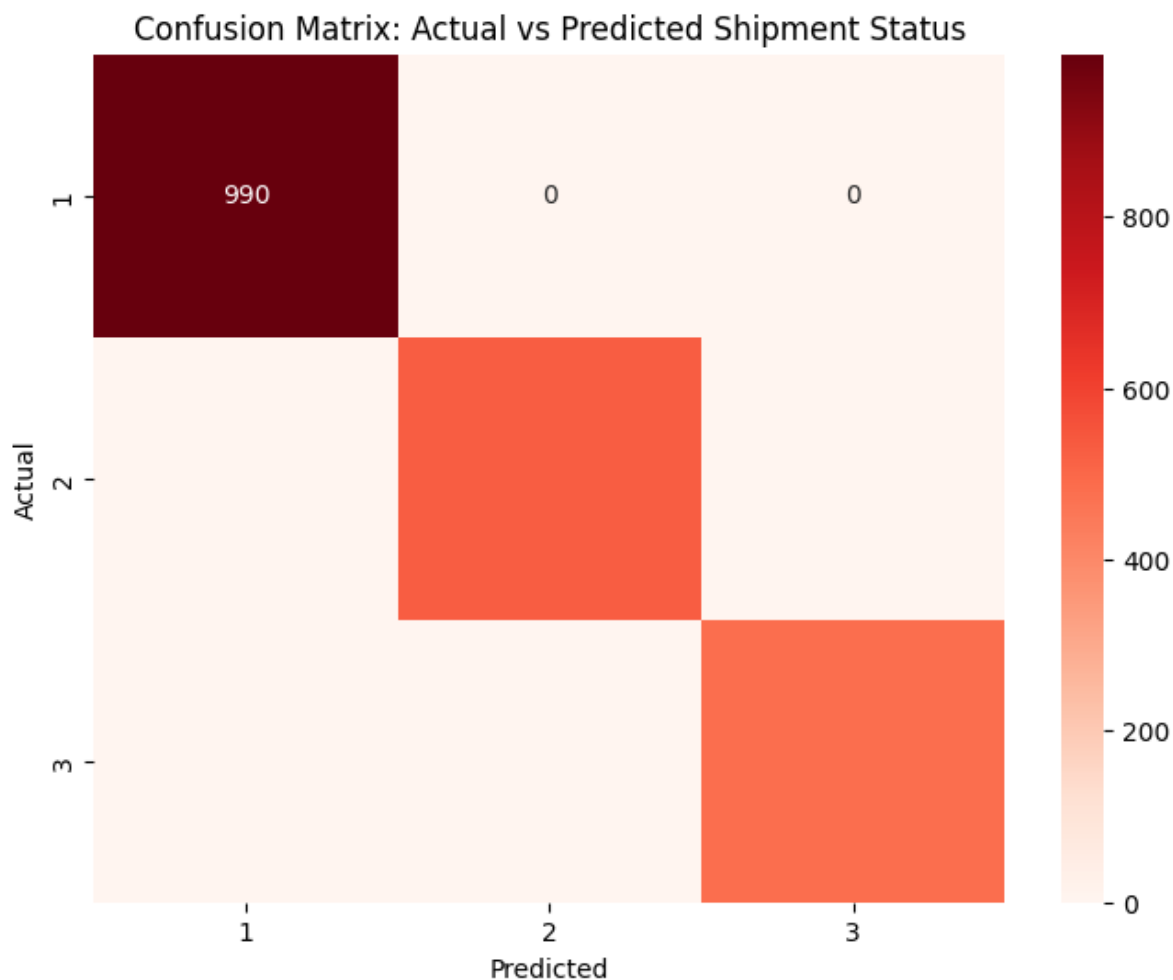
Accuracy: 0.9984992496248124

## Graphs and visualizations

```
In [59]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

comparison_df = pd.DataFrame({'Actual': Y_test, 'Predicted': y_pred})
from sklearn.metrics import confusion_matrix
conf_matrix = confusion_matrix(Y_test, y_pred)

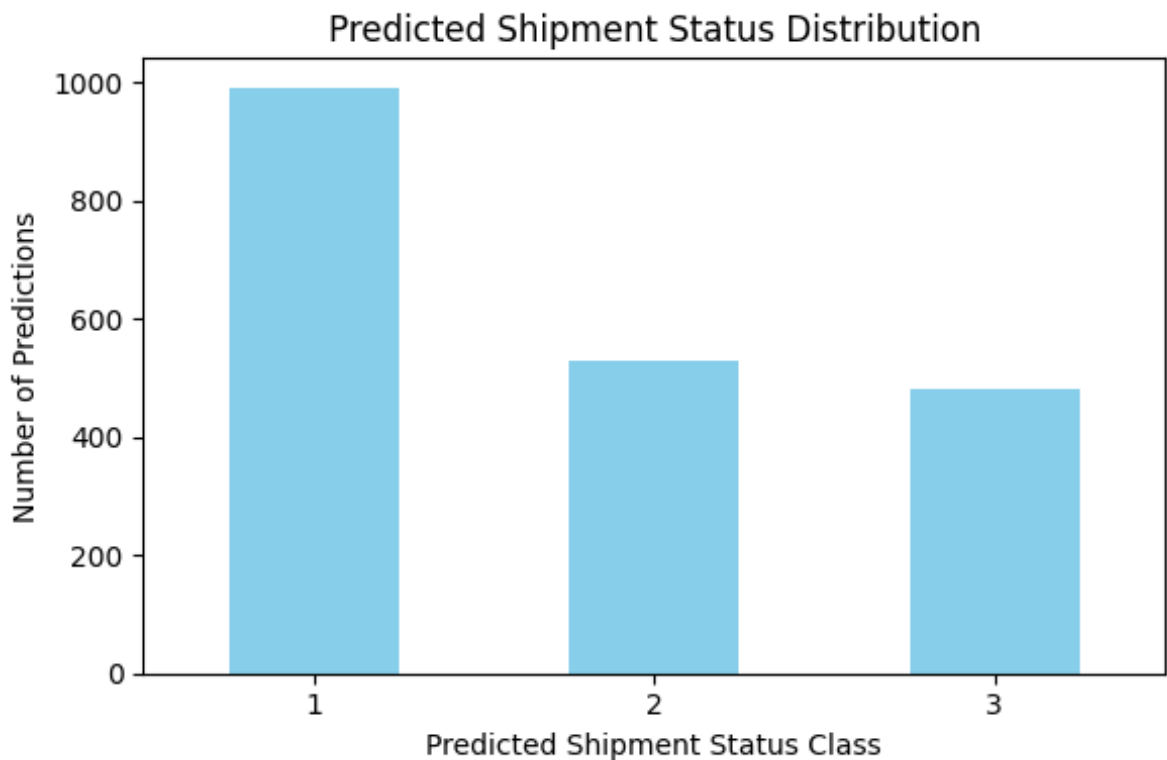
# Plot the confusion matrix as a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Reds', xticklabels=np.unique(Y), yticklabels=np.unique(Y))
plt.title("Confusion Matrix: Actual vs Predicted Shipment Status")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
In [60]: import matplotlib.pyplot as plt

# Count how many predictions fell into each class
prediction_counts = pd.Series(y_pred).value_counts().sort_index()

# Plot as a bar chart
plt.figure(figsize=(6, 4))
prediction_counts.plot(kind='bar', color='skyblue')
plt.title("Predicted Shipment Status Distribution")
plt.xlabel("Predicted Shipment Status Class")
plt.ylabel("Number of Predictions")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



### ***Brief explanation of findings***

1. ShipMode\_encoded has the strongest influence on shipment status.
2. Features like Profit, Region\_encoded, and ProductName\_encoded show weak or neutral correlation.
3. Class 3 has lower performance, suggesting a class imbalance or overlapping features.

```
'Shipped Early': 1,  
'Shipped Late': 2,  
'Shipped On Time': 3
```

- Most shipments were predicted as “Shipped Early”
- A moderate number as “Shipped Late”
- The least number as “Shipped On Time”

## ***5.2 Predict Sales Forecast***

***Data pulled from warehouse***

```
In [61]: query2 = """
WITH t1 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERDATE) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMTIME
),
t2 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY SHIPSTATUS_ENCODED) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMSHIPPING
),
t3 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY PRODUCTNAME) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMPRODUCT
),
t4 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY CITY) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMCUSTOMER
),
t5 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERID) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.FACTSALES
)

SELECT
    t1.ORDERDATE,
    t1.SHIPDATE,
    t2.SHIPSTATUS_ENCODED,
    t2.SHIPMODE_ENCODED,
    t2.DAYSTOSHIPSCHEDULED,
    t2.DAYSTOSHIPACTUAL,
    t3.PRODUCTNAME,
    t3.CATEGORY_ENCODED,
    t3.SUB_CATEGORY_ENCODED,
    t4.CITY,
    t4.LATITUDE,
    t4.LONGITUDE,
    t4.POSTALCODE,
    t4.STATE,
    t4.REGION_ENCODED,
    t4.SEGMENT_ENCODED,
    t5.DISCOUNT,
    t5.PROFIT,
    t5.PROFITRATIO,
    t5.QUANTITY,
    t5.SALES,
    t5.SALESPERCUSTOMER,
    t5.SALESFORECAST
FROM
    t5
JOIN t2 ON t5.rn = t2.rn
JOIN t3 ON t5.rn = t3.rn
JOIN t4 ON t5.rn = t4.rn
JOIN t1 ON t5.rn = t1.rn;

"""
```



```
In [62]: df_q2 = pd.read_sql(query2, conn)
```

```
/var/folders/4k/wvlpxp711yz8cxnbyqjw1zlh0000gn/T/ipykernel_38808/1206029902.py:1: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
```

```
df_q2 = pd.read_sql(query2, conn)
```

```
In [63]: df_q2.head()
```

Out[63]:

|   | ORDERDATE  | SHIPDATE   | SHIPSTATUS_ENCODED | SHIPMODE_ENCODED | DAYSTOSHIPS |
|---|------------|------------|--------------------|------------------|-------------|
| 0 | 2011-01-04 | 2011-01-08 | 1                  |                  | 1           |
| 1 | 2011-01-05 | 2011-01-09 | 1                  |                  | 1           |
| 2 | 2011-01-05 | 2011-01-09 | 1                  |                  | 1           |
| 3 | 2011-01-05 | 2011-01-09 | 1                  |                  | 1           |
| 4 | 2011-01-06 | 2011-01-13 | 1                  |                  | 1           |

5 rows × 23 columns

### ***Feature engineering and preprocessing***

in above df\_q1 dataframe has importance categorical data. First we should convert those into numerical using label encoding

```
In [64]: from sklearn.preprocessing import LabelEncoder

le_city = LabelEncoder()
df_q2['CITY_encoded'] = le_city.fit_transform(df_q2['CITY'])

le_productname = LabelEncoder()
df_q2['PRODUCTNAME_encoded'] = le_city.fit_transform(df_q2['PRODUCTNAME'])

le_state = LabelEncoder()
df_q2['STATE_encoded'] = le_city.fit_transform(df_q2['STATE'])
```

```
In [65]: df_q2.head()
```

```
Out[65]:
```

|   | ORDERDATE  | SHIPDATE   | SHIPSTATUS_ENCODED | SHIPMODE_ENCODED | DAYSTOSHIPS |
|---|------------|------------|--------------------|------------------|-------------|
| 0 | 2011-01-04 | 2011-01-08 | 1                  | 1                |             |
| 1 | 2011-01-05 | 2011-01-09 | 1                  | 1                |             |
| 2 | 2011-01-05 | 2011-01-09 | 1                  | 1                |             |
| 3 | 2011-01-05 | 2011-01-09 | 1                  | 1                |             |
| 4 | 2011-01-06 | 2011-01-13 | 1                  | 1                |             |

5 rows × 26 columns

```
In [66]: #drop unnecassary categorical columns  
df_q2.drop(['PRODUCTNAME', 'CITY', 'STATE'], axis=1, inplace=True)
```

```
In [67]: df_q2.head()
```

```
Out[67]:
```

|   | ORDERDATE  | SHIPDATE   | SHIPSTATUS_ENCODED | SHIPMODE_ENCODED | DAYSTOSHIPS |
|---|------------|------------|--------------------|------------------|-------------|
| 0 | 2011-01-04 | 2011-01-08 | 1                  | 1                |             |
| 1 | 2011-01-05 | 2011-01-09 | 1                  | 1                |             |
| 2 | 2011-01-05 | 2011-01-09 | 1                  | 1                |             |
| 3 | 2011-01-05 | 2011-01-09 | 1                  | 1                |             |
| 4 | 2011-01-06 | 2011-01-13 | 1                  | 1                |             |

5 rows × 23 columns

```
In [68]: df_q2.shape
```

```
Out[68]: (9993, 23)
```

## Data mining techniques, evaluation

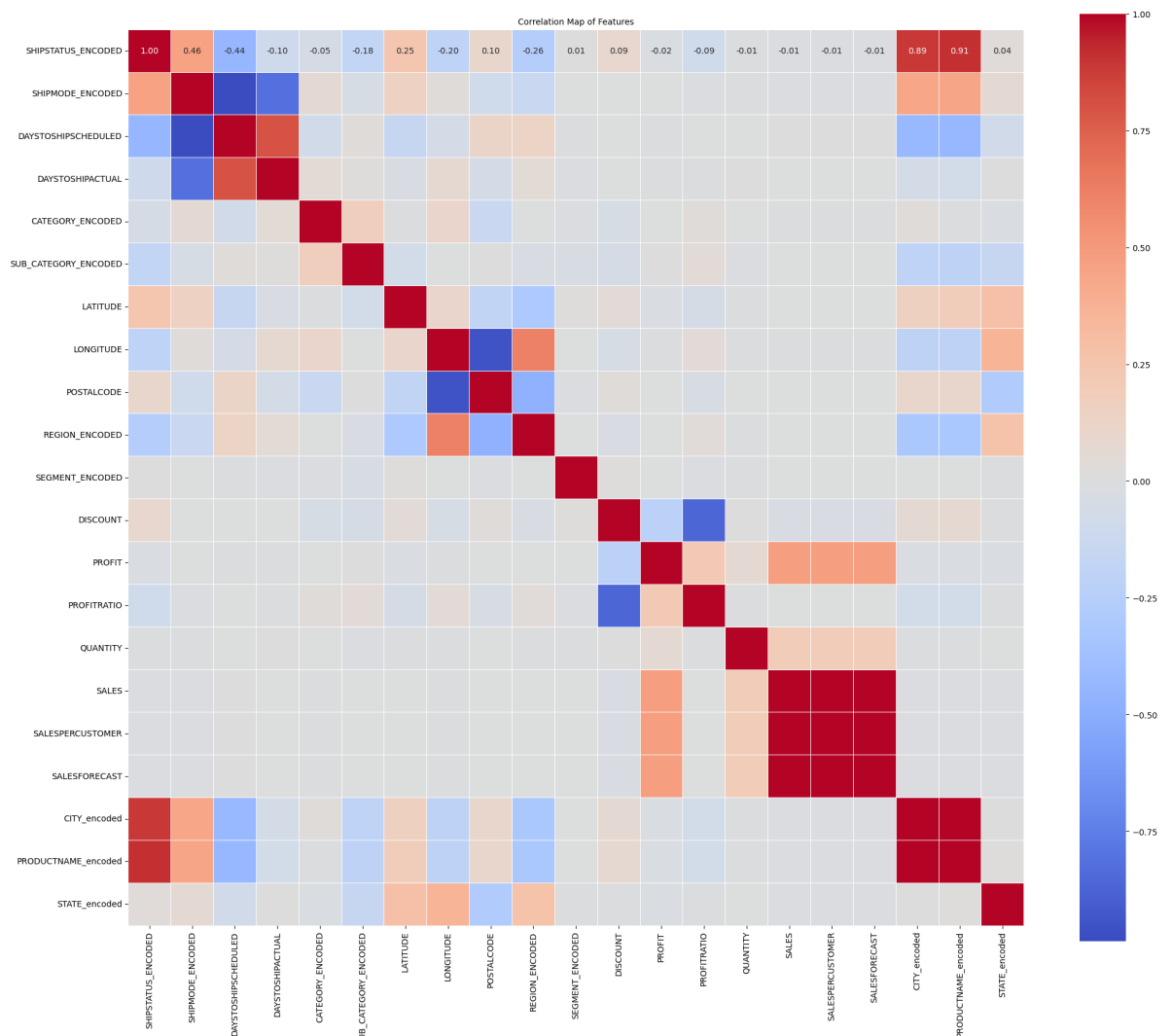
```
In [69]: df_q2_copy = df_q2.copy() #make copy from original
```

```
In [70]: df_q2_copy.drop(['ORDERDATE', 'SHIPDATE'], axis=1, inplace=True)
```

```
In [71]: import matplotlib.pyplot as plt
import seaborn as sns

corr_matrix = df_q2_copy.corr()

plt.figure(figsize=(24, 20))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm', square=True, linewidths=0.5)
plt.title("Correlation Map of Features", fontsize=10)
plt.show()
```



According to this correlation map . we have to remove following columns

```
In [72]: dropCols = [
    'CATEGORY_ENCODED', 'SUB_CATEGORY_ENCODED', 'LATITUDE', 'LONGITUDE',
    'POSTALCODE', 'CITY_encoded', 'STATE_encoded',
    'PRODUCTNAME_encoded', 'DAYSTOSHIPSCCHEDULED',
    'DAYSTOSHIPACTUAL', 'SHIPSTATUS_ENCODED', 'SHIPMODE_ENCODED', 'REGION_ENCODED', 'SEGMENT_ENCODED'
]
```

```
In [73]: df_q2_copy.drop(dropCols,axis=1,inplace=True)
```

```
In [74]: df_q2_copy.head()
```

Out[74]:

|   | DISCOUNT | PROFIT | PROFITRATIO | QUANTITY | SALES | SALESPERCUSTOMER | SALESF |
|---|----------|--------|-------------|----------|-------|------------------|--------|
| 0 | 0.0      | 110.0  | 29.0        | 3        | 378.0 | 377.97           |        |
| 1 | 0.2      | -88.0  | -17.5       | 3        | 502.0 | 502.49           |        |
| 2 | 0.2      | 69.0   | 35.0        | 6        | 197.0 | 196.70           |        |
| 3 | 0.2      | 32.0   | 35.0        | 6        | 91.0  | 91.06            |        |
| 4 | 0.2      | 1.0    | 33.8        | 1        | 4.0   | 3.93             |        |

```
In [75]: Y = df_q2_copy['SALESFORECAST'] #CREATE target variable
X = df_q2_copy.drop(['SALESFORECAST'], axis=1) #create features
```

```
In [76]: from sklearn.model_selection import train_test_split

#splitting dataset into train and test(20%)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
```

```
In [77]: from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor()
model.fit(X_train, Y_train)
```

Out[77]:

▼ RandomForestRegressor



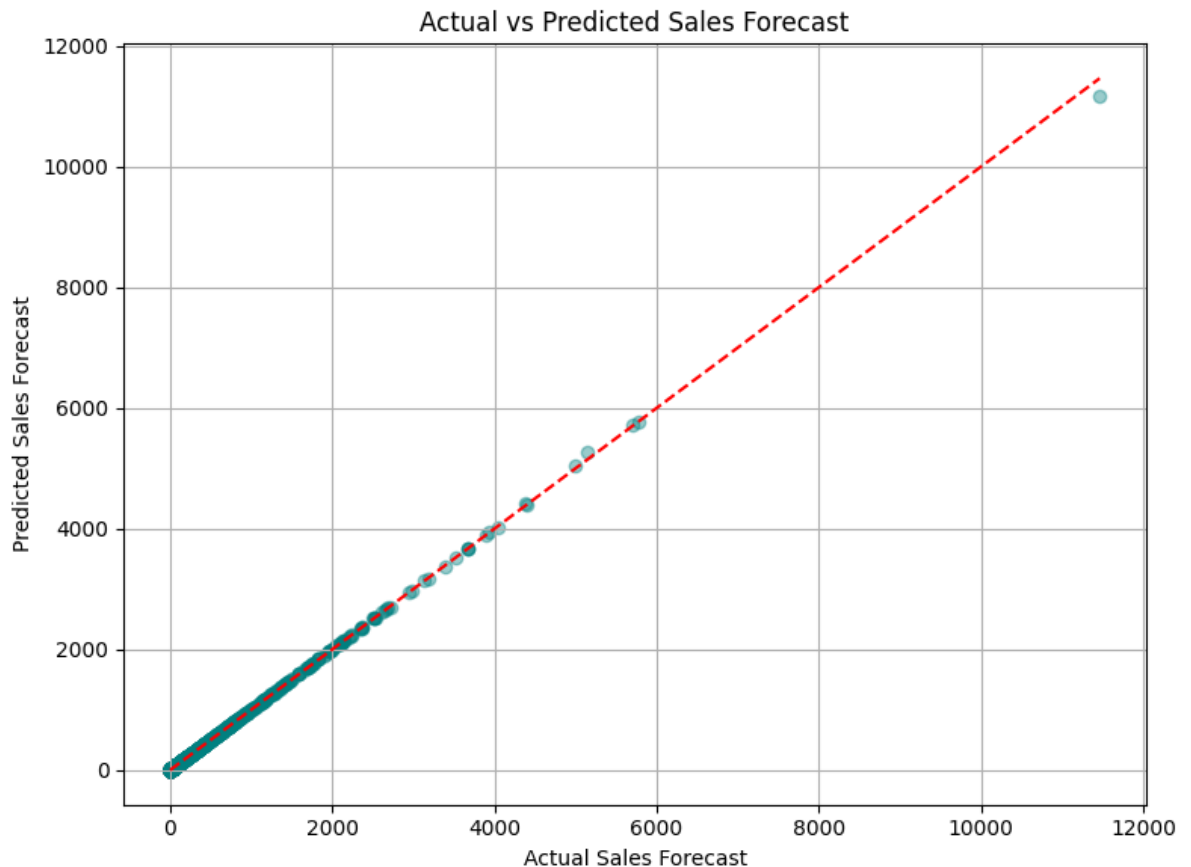
RandomForestRegressor()

(<https://scikit-learn.org/1.6/modules/generated/sklearn.ensemble.RandomForestRegressor.html>)

```
In [78]: y_pred = model.predict(X_test)
```

```
In [79]: import matplotlib.pyplot as plt

# Plot actual vs predicted values
plt.figure(figsize=(8, 6))
plt.scatter(Y_test, y_pred, alpha=0.4, color='teal')
plt.plot([Y_test.min(), Y_test.max()], [Y_test.min(), Y_test.max()], 'r--')
plt.xlabel('Actual Sales Forecast')
plt.ylabel('Predicted Sales Forecast')
plt.title('Actual vs Predicted Sales Forecast')
plt.grid(True)
plt.tight_layout()
plt.show()
```



### Evaluation with rmse and r-squared

```
In [80]: from sklearn.metrics import root_mean_squared_error, r2_score

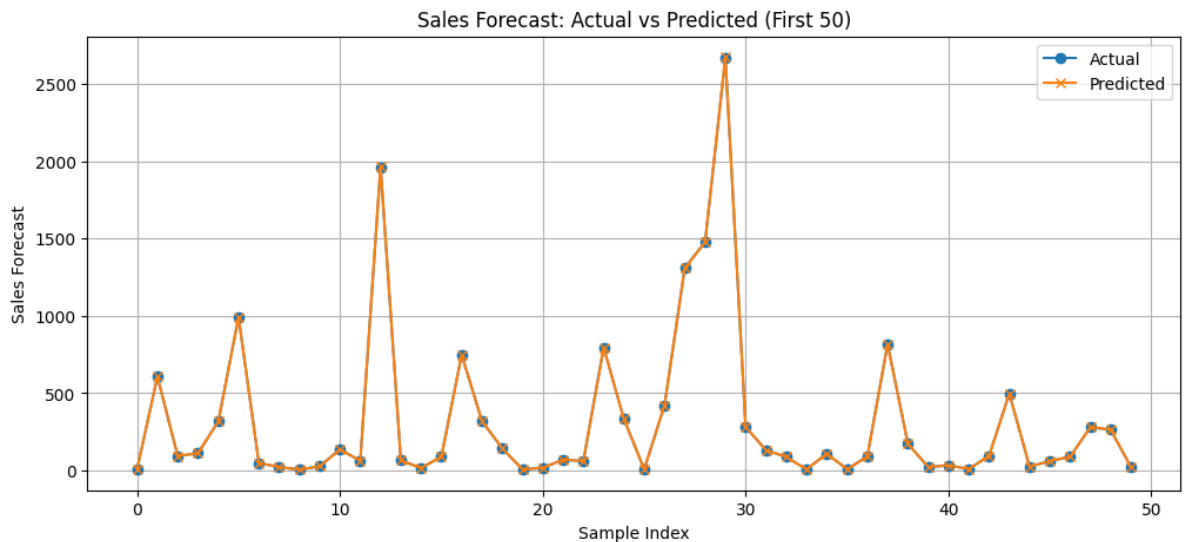
rmse = root_mean_squared_error(Y_test, y_pred)
r2 = r2_score(Y_test, y_pred)

print("root mean square error: ", rmse)
print("r-squared: ", r2)
```

```
root mean square error: 7.592584571161184
r-squared: 0.9998439814090101
```

## Graphs and visualizations

```
In [81]: plt.figure(figsize=(12, 5))
plt.plot(Y_test[:50].values, label='Actual', marker='o')
plt.plot(y_pred[:50], label='Predicted', marker='x')
plt.title('Sales Forecast: Actual vs Predicted (First 50)')
plt.xlabel('Sample Index')
plt.ylabel('Sales Forecast')
plt.legend()
plt.grid(True)
plt.show()
```



```
In [82]: from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestRegressor
import numpy as np

# Initialize the model
model = RandomForestRegressor(n_estimators=100, random_state=42)

# Perform 5-fold cross-validation using negative RMSE
cv_scores = cross_val_score(
    model,
    X, Y, cv=5, scoring='neg_root_mean_squared_error'
)

cv_rmse_scores = -cv_scores
avg_rmse = np.mean(cv_rmse_scores)

print("Cross-Validation RMSEs:", cv_rmse_scores)
print("Average RMSE:", avg_rmse)

Cross-Validation RMSEs: [261.16193423  12.58291787  81.20647655  4
 5.17165291  26.90125612]
Average RMSE: 85.4048475373217
```

## Brief explanation of findings

This sales forecasting model offers a powerful tool for decision-makers to predict and manage future sales. With minimal input and high precision, it supports better budgeting, smarter promotions, and efficient inventory planning—delivering a competitive business advantage

### 5.3 Predict Order Profitability

```
In [83]: query3 = """
WITH t1 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERDATE) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMTIME
),
t2 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY SHIPSTATUS_ENCODED) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMSHIPPING
),
t3 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY PRODUCTNAME) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMPRODUCT
),
t4 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY CITY) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMCUSTOMER
),
t5 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERID) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.FACTSALES
)

SELECT
    t2.SHIPSTATUS_ENCODED,
    t2.SHIPMODE_ENCODED,
    t2.DAYSTOSHIPSCHEDULED,
    t2.DAYSTOSHIPACTUAL,
    t3.CATEGORY_ENCODED,
    t3.SUB_CATEGORY_ENCODED,
    t4.REGION_ENCODED,
    t4.SEGMENT_ENCODED,
    t5.DISCOUNT,
    t5.PROFIT,
    t5.PROFITRATIO,
    t5.QUANTITY,
    t5.SALES,
    t5.SALESPERCUSTOMER,
FROM
    t5
JOIN t2 ON t5.rn = t2.rn
JOIN t3 ON t5.rn = t3.rn
JOIN t4 ON t5.rn = t4.rn
JOIN t1 ON t5.rn = t1.rn;

"""
```

```
In [84]: df_q3 = pd.read_sql(query3, conn)
```

```
/var/folders/4k/wvlpxp711yz8cxnbyqjw1zlh0000gn/T/ipykernel_38808/4
92894181.py:1: UserWarning: pandas only supports SQLAlchemy connec
table (engine/connection) or database string URI or sqlite3 DBAPI2
connection. Other DBAPI2 objects are not tested. Please consider u
sing SQLAlchemy.
```

```
df_q3 = pd.read_sql(query3, conn)
```

```
In [85]: df_q3.head()
```

Out[85]:

|   | SHIPSTATUS_ENCODED | SHIPMODE_ENCODED | DAYSTOSHIPSCCHEDULED | DAYSTOSHIP/ |
|---|--------------------|------------------|----------------------|-------------|
| 0 | 1                  | 1                | 6                    |             |
| 1 | 1                  | 1                | 6                    |             |
| 2 | 1                  | 1                | 6                    |             |
| 3 | 1                  | 1                | 6                    |             |
| 4 | 1                  | 1                | 6                    |             |

### ***Feature engineering and preprocessing***



```
In [86]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Columns to analyze
columns_to_check = ['PROFIT', 'SALES', 'DISCOUNT']

for col in columns_to_check:
    # Calculate IQR
    Q1 = df_q3[col].quantile(0.25)
    Q3 = df_q3[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Identify outliers
    outliers = df_q3[(df_q3[col] < lower_bound) | (df_q3[col] > upper_bound)]

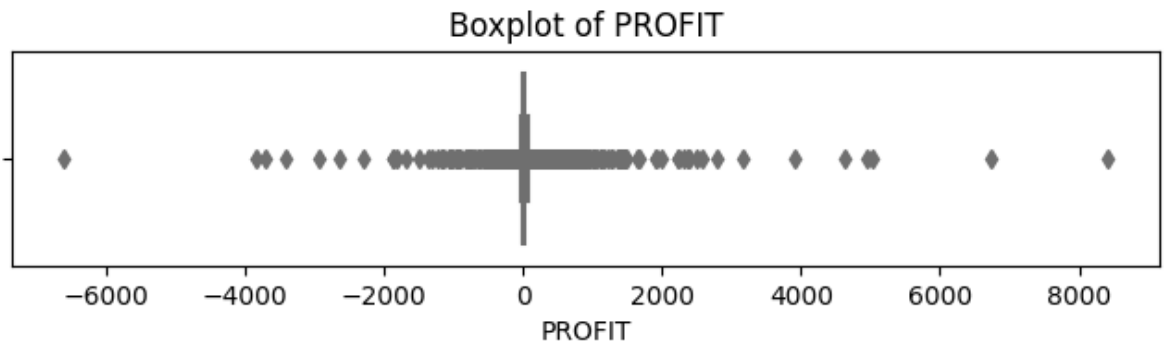
    print(f"Column: {col}")
    print(f"Number of outliers: {len(outliers)}")
    print("-" * 30)

    # Plot boxplot
    plt.figure(figsize=(8, 1.5))
    sns.boxplot(x=df_q3[col], color='skyblue')
    plt.title(f'Boxplot of {col}')
    plt.show()
```

Column: PROFIT

Number of outliers: 1924

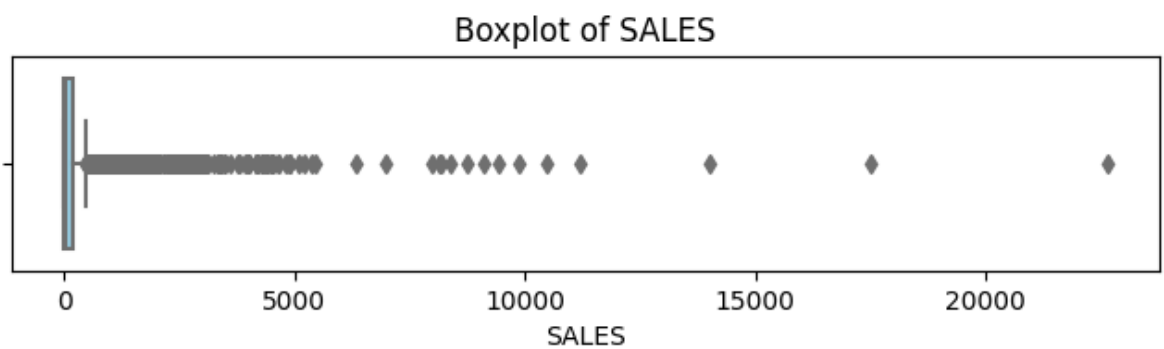
-----



Column: SALES

Number of outliers: 1166

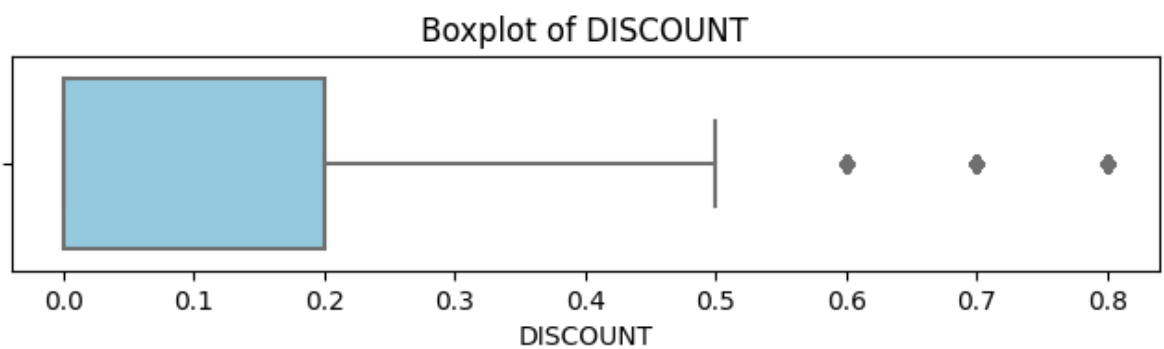
-----



Column: DISCOUNT

Number of outliers: 856

-----



```
In [87]: df_q3_copy = df_q3.copy()
```

```
In [88]: df_q3_copy = df_q3_copy[(df_q3_copy[col] >= lower_bound) & (df_q3_copy[col] <= upper_bound)]
```

```
In [89]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Columns to analyze
columns_to_check = ['PROFIT', 'SALES', 'DISCOUNT']

for col in columns_to_check:
    # Calculate IQR
    Q1 = df_q3_copy[col].quantile(0.25)
    Q3 = df_q3_copy[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Identify outliers
    outliers = df_q3_copy[(df_q3_copy[col] < lower_bound) | (df_q3_
copy[col] > upper_bound)]

    print(f"Column: {col}")
    print(f"Number of outliers: {len(outliers)}")
    print("-" * 30)

    # Plot boxplot
    plt.figure(figsize=(8, 1.5))
    sns.boxplot(x=df_q3_copy[col], color='skyblue')
    plt.title(f'Boxplot of {col}')
    plt.show()
```

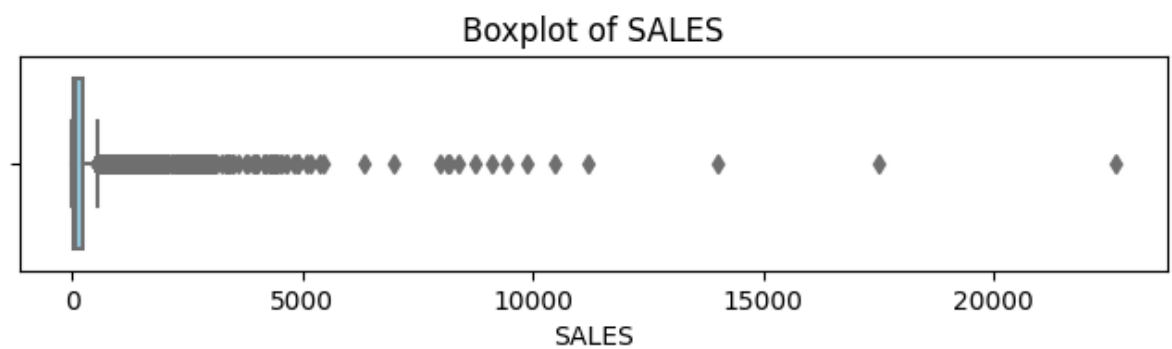
Column: PROFIT

Number of outliers: 1508



Column: SALES

Number of outliers: 1020



Column: DISCOUNT

Number of outliers: 0



```
In [90]: df_q3 = df_q3_copy
```

```
In [91]: df_q3.head()
```

```
Out[91]:
```

|   | SHIPSTATUS_ENCODED | SHIPMODE_ENCODED | DAYSTOSHIPSCHEDULED | DAYSTOSHIP/ |
|---|--------------------|------------------|---------------------|-------------|
| 0 | 1                  | 1                | 6                   |             |
| 1 | 1                  | 1                | 6                   |             |
| 2 | 1                  | 1                | 6                   |             |
| 3 | 1                  | 1                | 6                   |             |
| 4 | 1                  | 1                | 6                   |             |

```
In [92]: df_q3.shape
```

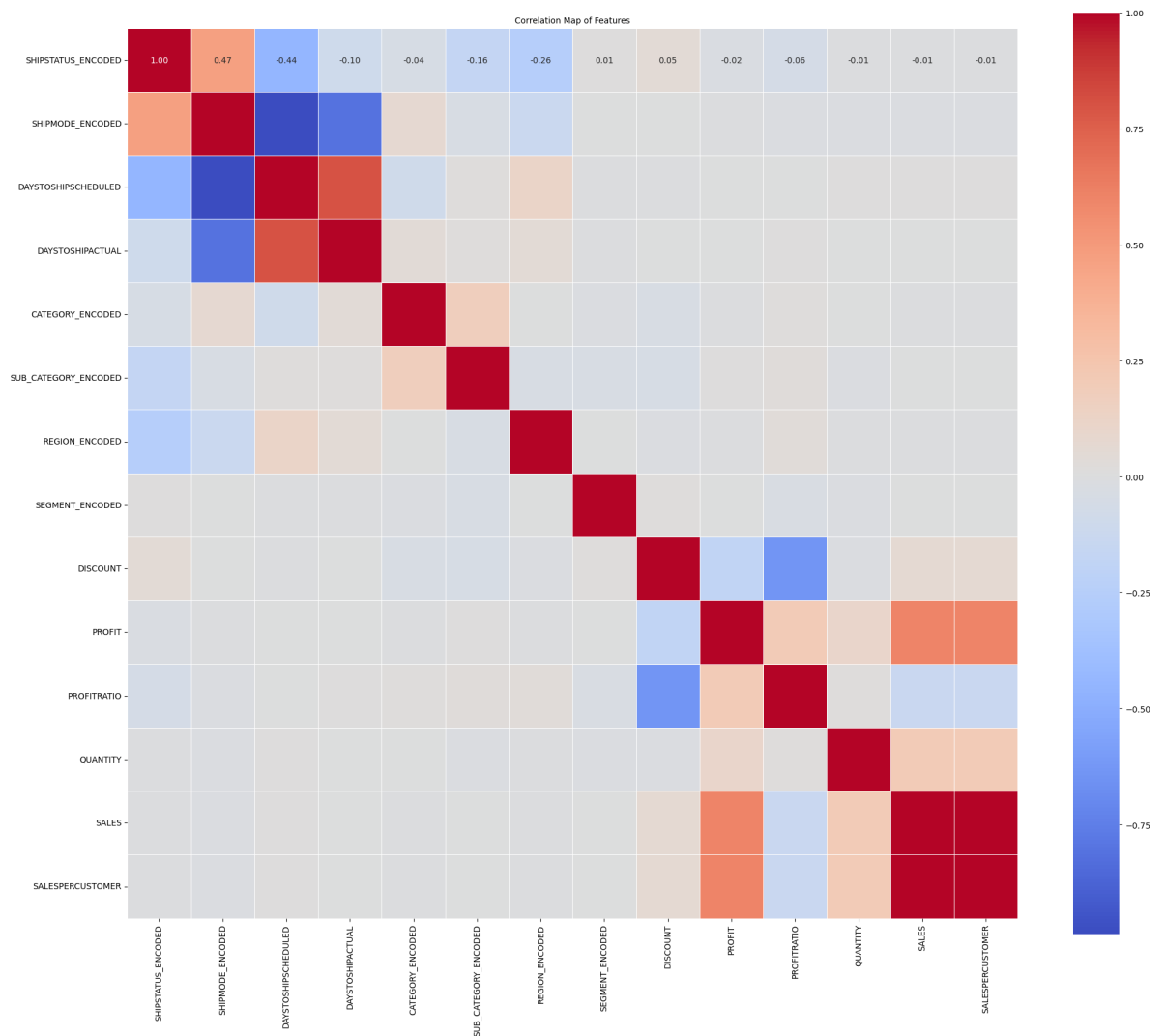
```
Out[92]: (9137, 14)
```

### ***Data mining techniques, evaluation***

```
In [93]: import matplotlib.pyplot as plt
import seaborn as sns

corr_matrix = df_q3.corr()

plt.figure(figsize=(24, 20))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm', square=True, linewidths=0.5)
plt.title("Correlation Map of Features", fontsize=10)
plt.show()
```



```
In [94]: dropCols = [ 'SALESPERCUSTOMER', 'DAYSTOSHIPACTUAL', 'SUB_CATEGORY_ENCODED' ]
df_q3.drop(dropCols,axis=1,inplace=True)
```

```
In [95]: df_q3.head()
```

```
Out[95]:
```

|   | SHIPSTATUS_ENCODED | SHIPMODE_ENCODED | DAYSTOSHIPSCHEMULED | CATEGORY_ENCODED |
|---|--------------------|------------------|---------------------|------------------|
| 0 |                    | 1                | 1                   | 6                |
| 1 |                    | 1                | 1                   | 6                |
| 2 |                    | 1                | 1                   | 6                |
| 3 |                    | 1                | 1                   | 6                |
| 4 |                    | 1                | 1                   | 6                |

```
In [96]: Y = df_q3[ 'PROFIT' ] #target variable
X = df_q3.drop([ 'PROFIT' ],axis=1) #create feature
```

**Evaluation with rmse and r-squard**

```
In [97]: from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import root_mean_squared_error, r2_score

# Split data
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

# Train model
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, Y_train)

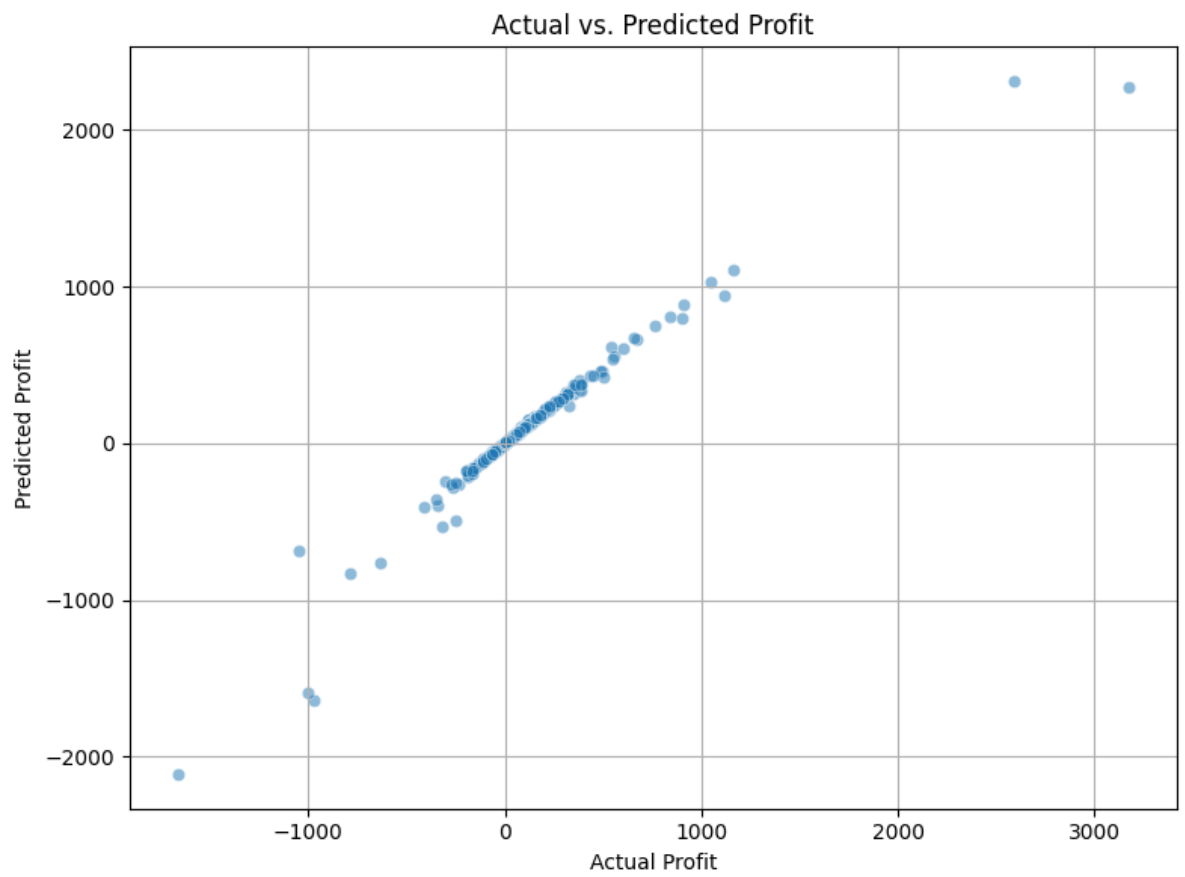
# Predict and evaluate
y_pred = model.predict(X_test)
rmse = root_mean_squared_error(Y_test, y_pred)
r2 = r2_score(Y_test, y_pred)

rmse, r2
```

```
Out[97]: (35.05790422926044, 0.9450298194341162)
```

## Graphs and visualizations

```
In [98]: # Plot predicted vs actual
plt.figure(figsize=(8, 6))
sns.scatterplot(x=Y_test, y=y_pred, alpha=0.5)
plt.xlabel("Actual Profit")
plt.ylabel("Predicted Profit")
plt.title("Actual vs. Predicted Profit")
plt.grid(True)
plt.tight_layout()
plt.show()
```



**5.4 Cluster Orders by Buying Behavior - Group orders by similar buying patterns**



```

In [99]: query4 = '''
WITH t1 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERDATE) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMTIME
),
t2 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY SHIPSTATUS_ENCODED) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMSHIPPING
),

t3 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY PRODUCTNAME) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMPRODUCT
),
t4 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY CITY) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMCUSTOMER
),
t5 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERID) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.FACTSALES
)

SELECT
    t3.CATEGORY_ENCODED,
    t3.SUB_CATEGORY_ENCODED,
    t4.SEGMENT_ENCODED,
    t5.DISCOUNT,
    t5.PROFIT,
    t5.PROFITRATIO,
    t5.QUANTITY,
    t5.SALES,
    t5.SALESPERCUSTOMER,
FROM
    t5
JOIN t2 ON t5.rn = t2.rn
JOIN t3 ON t5.rn = t3.rn
JOIN t4 ON t5.rn = t4.rn
JOIN t1 ON t5.rn = t1.rn;

'''

```

```

In [100]: df_q4 = pd.read_sql(query4, conn)

/var/folders/4k/wvlpxp71lyz8cxnbyqjw1zlh0000gn/T/ipykernel_38808/4
102763816.py:1: UserWarning: pandas only supports SQLAlchemy conne
ctable (engine/connection) or database string URI or sqlite3 DBAPI
2 connection. Other DBAPI2 objects are not tested. Please consider
using SQLAlchemy.
    df_q4 = pd.read_sql(query4, conn)

```

```
In [101]: df_q4.head()
```

Out[101]:

|   | CATEGORY_ENCODED | SUB_CATEGORY_ENCODED | SEGMENT_ENCODED | DISCOUNT | F |
|---|------------------|----------------------|-----------------|----------|---|
| 0 | 1                | 12                   | 1               | 0.0      |   |
| 1 | 1                | 12                   | 1               | 0.2      |   |
| 2 | 1                | 12                   | 1               | 0.2      |   |
| 3 | 1                | 12                   | 2               | 0.2      |   |
| 4 | 1                | 12                   | 1               | 0.2      |   |

### ***Feature engineering and preprocessing***

```
In [102]: from sklearn.preprocessing import StandardScaler
X_cluster = df_q4

# Normalize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_cluster)
```

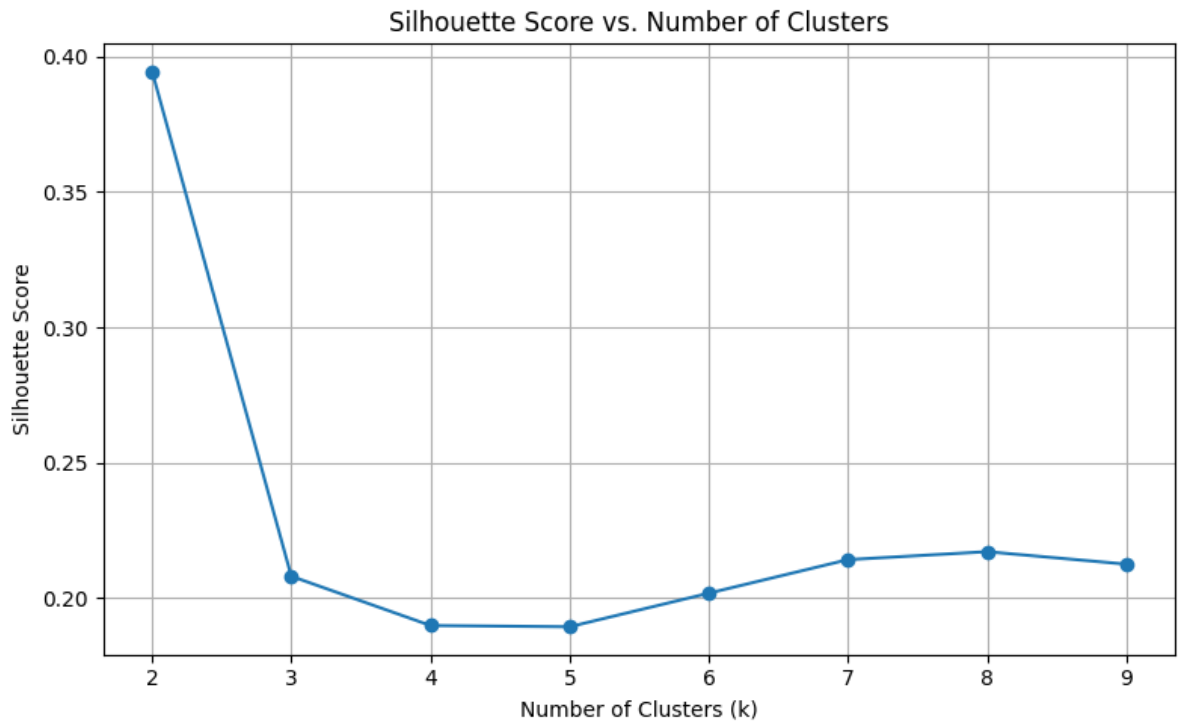
### ***Data mining techniques, evaluation***

```
In [103]: from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import seaborn as sns

# Find optimal number of clusters using silhouette score
silhouette_scores = []
k_range = range(2, 10)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    score = silhouette_score(X_scaled, kmeans.labels_)
    silhouette_scores.append(score)
```

### ***Evaluation with silhouette\_score***

```
In [104]: # Plot silhouette scores to find the best k
plt.figure(figsize=(8, 5))
plt.plot(k_range, silhouette_scores, marker='o')
plt.title("Silhouette Score vs. Number of Clusters")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Silhouette Score")
plt.grid(True)
plt.tight_layout()
plt.show()
```



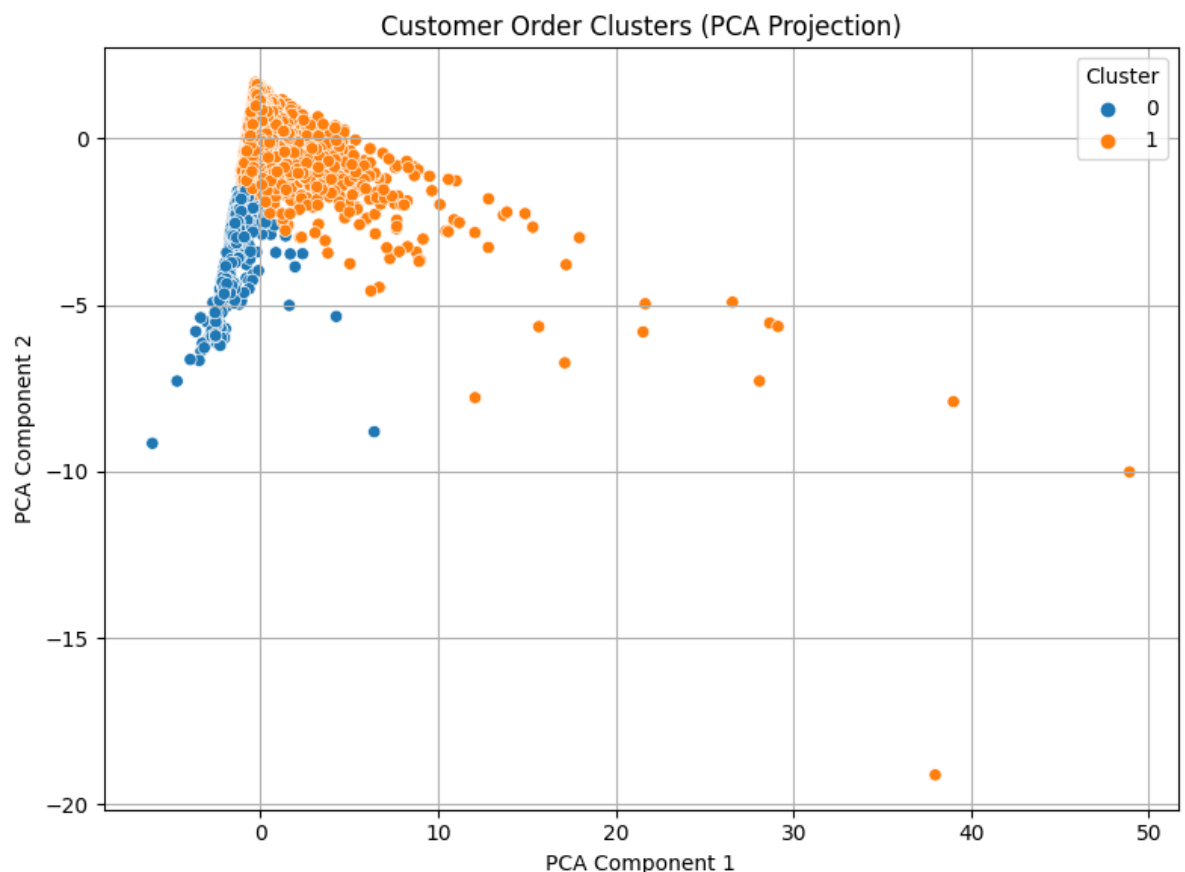
according to silhouette score, optimal\_k is 2

```
In [105]: from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

optimal_k = 2
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
df_q4['Cluster'] = kmeans.fit_predict(X_scaled)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
df_q4['PCA1'] = X_pca[:, 0]
df_q4['PCA2'] = X_pca[:, 1]

# Visualize
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df_q4, x='PCA1', y='PCA2', hue='Cluster', palette='tab10')
plt.title("Customer Order Clusters (PCA Projection)")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.grid(True)
plt.tight_layout()
plt.show()
```



## 5.5 Visualize shipment performance across geographies

```
In [106]: query5 = """
WITH t1 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERDATE) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMTIME
),
t2 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY SHIPSTATUS_ENCODED) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMSHIPPING
),

t3 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY PRODUCTNAME) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMPRODUCT
),
t4 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY CITY) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMCUSTOMER
),
t5 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERID) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.FACTSALES
)

SELECT
    t2.SHIPSTATUS_ENCODED,
    t2.SHIPMODE_ENCODED,
    t2.DAYSTOSHIPSCHEDULED,
    t2.DAYSTOSHIPACTUAL,
    t3.CATEGORY_ENCODED,
    t3.SUB_CATEGORY_ENCODED,
    t4.LATITUDE,
    t4.LONGITUDE,
    t4.STATE
FROM
    t5
JOIN t2 ON t5.rn = t2.rn
JOIN t3 ON t5.rn = t3.rn
JOIN t4 ON t5.rn = t4.rn
JOIN t1 ON t5.rn = t1.rn;

"""
```

```
In [107]: df_q5 = pd.read_sql(query5, conn)

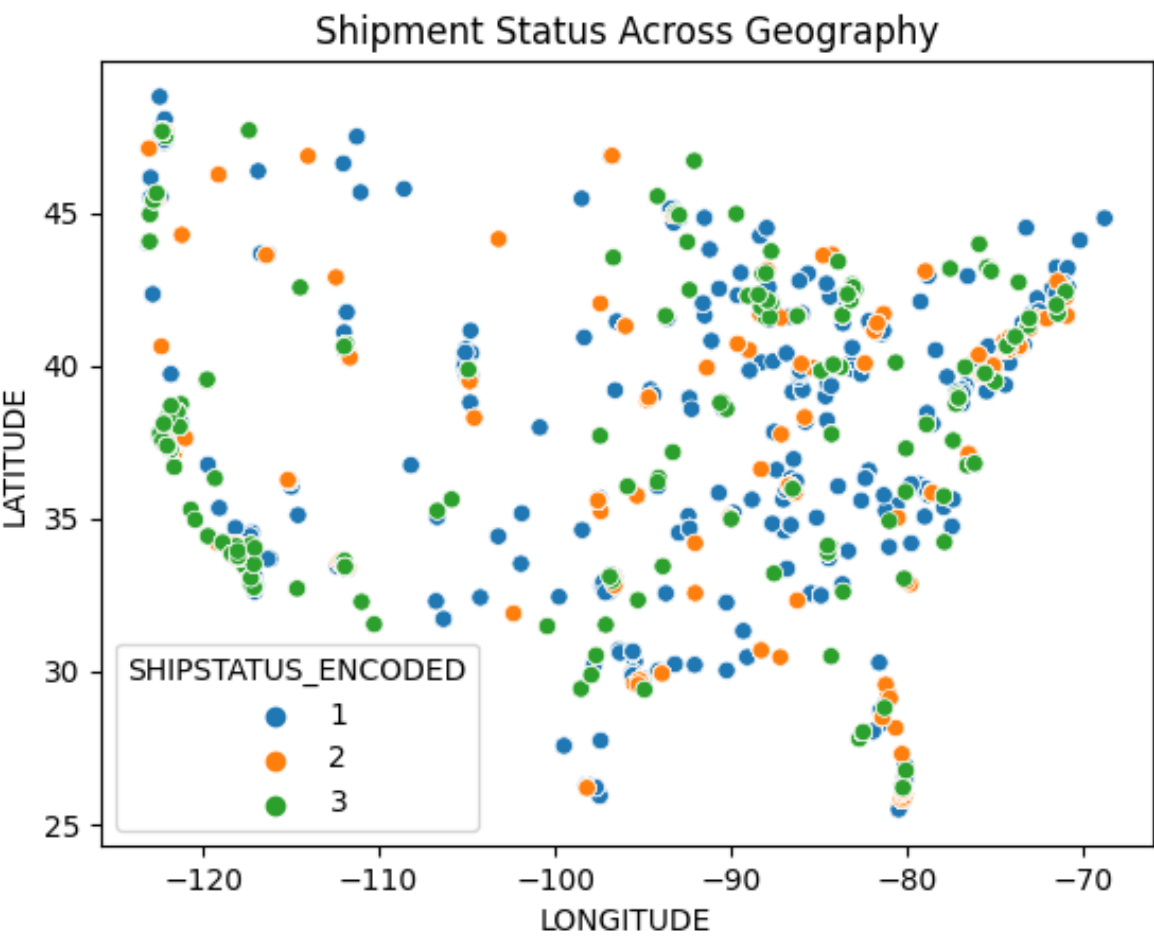
/var/folders/4k/wvlpxp71lyz8cxnbyqjw1zlh0000gn/T/ipykernel_38808/2
436508198.py:1: UserWarning: pandas only supports SQLAlchemy conne
ctable (engine/connection) or database string URI or sqlite3 DBAPI
2 connection. Other DBAPI2 objects are not tested. Please consider
using SQLAlchemy.
    df_q5 = pd.read_sql(query5, conn)
```

```
In [108]: df_q5.head()
```

Out[108]:

|   | SHIPSTATUS_ENCODED | SHIPMODE_ENCODED | DAYSTOSHIPSCHEDULED | DAYSTOSHIP/ |
|---|--------------------|------------------|---------------------|-------------|
| 0 | 1                  | 1                | 6                   |             |
| 1 | 1                  | 1                | 6                   |             |
| 2 | 1                  | 1                | 6                   |             |
| 3 | 1                  | 1                | 6                   |             |
| 4 | 1                  | 1                | 6                   |             |

```
In [109]: sns.scatterplot(data=df_q5, x='LONGITUDE', y='LATITUDE', hue='SHIPS
STATUS_ENCODED')
plt.title('Shipment Status Across Geography')
plt.show()
```



5.6 Analyze delivery delays or profit trends by latitude/longitude

Data pulled from warehouse

```

In [110]: query6 = """
WITH t1 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERDATE) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMTIME
),
t2 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY SHIPSTATUS_ENCODED) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMSHIPPING
),
t3 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY PRODUCTNAME) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMPRODUCT
),
t4 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY CITY) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.DIMCUSTOMER
),
t5 AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY ORDERID) AS rn
    FROM ASSIGNMENT_DB.ASSIGNMENT_SCHEMA.FACTSALES
)

SELECT
    t2.DAYSTOSHIPSCHEDULED,
    t2.DAYSTOSHIPACTUAL,
    t4.CITY,
    t4.LATITUDE,
    t4.LONGITUDE,
    t4.STATE,
    t5.ORDERID,
    t5.DISCOUNT,
    t5.PROFIT,
    t5.QUANTITY,
    t5.SALES
FROM
    t5
JOIN t2 ON t5.rn = t2.rn
JOIN t3 ON t5.rn = t3.rn
JOIN t4 ON t5.rn = t4.rn
JOIN t1 ON t5.rn = t1.rn;

"""

```

```

In [111]: df_q6 = pd.read_sql(query6,conn)

/var/folders/4k/wvlpxp71lyz8cxnbyqjwlzlh0000gn/T/ipykernel_38808/1861922320.py:1: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
  df_q6 = pd.read_sql(query6,conn)

```

```
In [112]: df_q6.head()
```

```
Out[112]:
```

|   | DAYSTOSHIPSCHEДУLED | DAYSTOSHIPACTUAL | CITY     | LATITUDE | LONGITUDE | STA       |
|---|---------------------|------------------|----------|----------|-----------|-----------|
| 0 | 6                   | 5                | Aberdeen | 45.4661  | -98.4856  | So<br>Dak |
| 1 | 6                   | 5                | Abilene  | 32.4320  | -99.7724  | Te        |
| 2 | 6                   | 5                | Akron    | 41.0334  | -81.4385  | O         |
| 3 | 6                   | 5                | Akron    | 41.0334  | -81.4385  | O         |
| 4 | 6                   | 4                | Akron    | 41.0334  | -81.4385  | O         |

### Data mining techniques, evaluation

```
In [113]: import plotly.express as px

#Feature Engineering
df_q6['ShippingDelay'] = df_q6['DAYSTOSHIPACTUAL'] - df_q6['DAYSTOSHIPSCHEДУLED']

#Group by coordinates and calculate average delivery delay and profit
latlong_summary = df_q6.groupby(['LATITUDE', 'LONGITUDE']).agg({
    'ShippingDelay': 'mean',
    'PROFIT': 'mean',
    'ORDERID': 'count'
}).reset_index().rename(columns={
    'ShippingDelay': 'AvgShippingDelay',
    'PROFIT': 'AvgProfit',
    'ORDERID': 'OrderCount'
})

#Evaluation metrics (simple descriptive)
avg_delay = latlong_summary['AvgShippingDelay'].mean()
avg_profit = latlong_summary['AvgProfit'].mean()
```

### Graphs and visualizations



```
In [114]: # 1. Heat scatter of delivery delays
fig1 = px.scatter_geo(latlong_summary,
                      lat='LATITUDE',
                      lon='LONGITUDE',
                      color='AvgShippingDelay',
                      size='OrderCount',
                      hover_name='AvgShippingDelay',
                      title='Average Delivery Delay by Location',
                      projection='natural earth')

fig1.show()

# 2. Heat scatter of profit trends
fig2 = px.scatter_geo(latlong_summary,
                      lat='LATITUDE',
                      lon='LONGITUDE',
                      color='AvgProfit',
                      size='OrderCount',
                      hover_name='AvgProfit',
                      title='Average Profit by Location',
                      projection='natural earth')

fig2.show()

avg_delay, avg_profit
```

```
Out[114]: (-0.5310038511800921, 30.750377457196787)
```

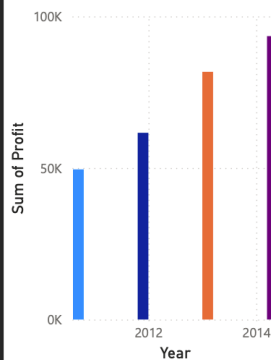
## Data visualization with power BI

dashboard link - <https://app.powerbi.com/groups/me/reports/f2f8b9ae-5fd4-4f35-9517-8f843fc752cc/307a28d0c082b02d05c5?experience=power-bi>  
(<https://app.powerbi.com/groups/me/reports/f2f8b9ae-5fd4-4f35-9517-8f843fc752cc/307a28d0c082b02d05c5?experience=power-bi>)

# Sales Analysis Dashboard

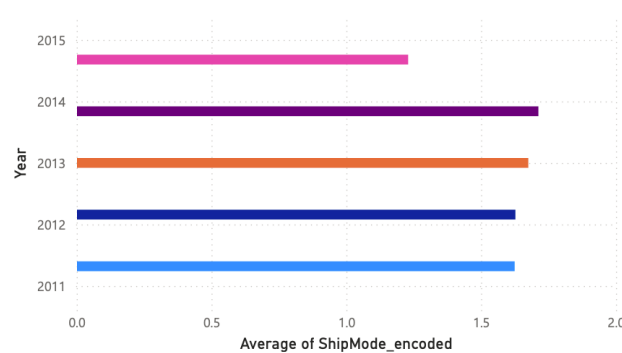
Sum of Profit by Year and Year

Year ● 2011 ● 2012 ● 2013 ● 2014



Average of ShipMode\_encoded by Year and Year

Year ● 2011 ● 2012 ● 2013 ● 2014 ● 2015



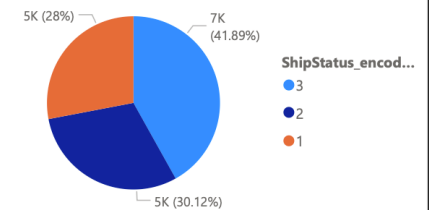
531

Count of City

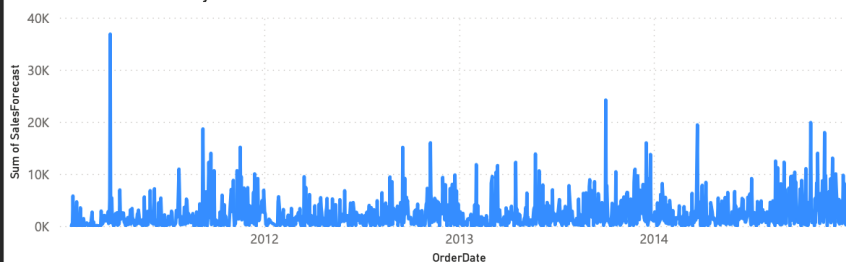
5009

Count of OrderID

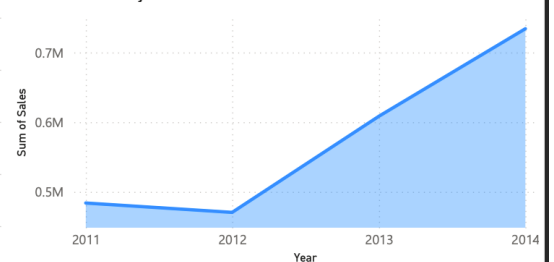
Sum of ShipStatus\_encoded by ShipStatus\_encoded



Sum of SalesForecast by OrderDate



Sum of Sales by Year



## Insights

### 1. Profit Growth Over Time

- The **sum of profit** steadily increased from **2011 to 2014**, with the most significant jump between **2012 and 2013**, and again from **2013 to 2014**.
- This indicates improving **operational efficiency** or better **pricing strategies** over the years.

### 2. Shipment Mode Trends

- The **average ShipMode\_encoded** value increased until **2014**, then **dropped in 2015**.
- This suggests a possible **shift to simpler or slower shipping modes** in 2015, or a change in how shipping is being categorized.

### 3. Geographic and Order Coverage

- **531 unique cities** and **5009 total orders** are recorded, indicating a **broad national presence** and **moderate order volume**.
- This scale allows for regional performance analysis.

### 4. Shipment Status Distribution

- **ShipStatus\_encoded = 3** (blue) makes up the largest portion at **~42%**
- The other two statuses ( **2** and **1** ) are almost evenly split (**~30% each**).